

**SCHOOL OF COMPUTER SCIENCE**  
**University of Petroleum and Energy Studies, Dehradun**  
**SUBJECT –CLOUD LAB**  
**LAB – REPORT**



**SUBMITTED BY**  
**Aditya Sharma**  
**SAP ID-500122015**

## **Lab-1**

### **Software and Hardware Requirements**

#### **Hardware**

- 64-bit system with virtualization support enabled in BIOS.
- Minimum 8 GB RAM (4 GB minimum acceptable).
- Internet Connection.

#### **Software (Windows Host)**

- Oracle VirtualBox
  - Vagrant
  - Windows Subsystem for Linux (WSL 2)
  - Ubuntu (WSL distribution)
  - Docker Engine (docker.io)
- 

## Theory

### Virtual Machine

A Virtual Machine emulates a complete physical computer, including its own operating system kernel, hardware drivers, and user space. Each VM runs on top of a hypervisor.

#### Characteristics:

- Full OS per VM
- Higher resource usage
- Strong isolation
- Slower startup time

### Container

Containers virtualize at the operating system level. They share the host OS kernel while isolating applications and dependencies in user space.

#### Characteristics:

- Shared kernel
- Lightweight
- Fast startup
- Efficient resource usage

## • Part A: Install Vagrant And automate VM:

1- Install Oracle Vm-

## 2- Download vagrant

## 3- Run vagrant to automate the VM

```
PS D:\container> vagrant -v
Vagrant 2.4.9
PS D:\container> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
PS D:\container> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
default: Box Provider: virtualbox
default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/providers/virtualbox/unknown/vagrant.box
default:
==> default: Successfully added box 'hashicorp/bionic64' (v1.0.282) for 'virtualbox'!
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to date...
==> default: Setting the name of the VM: container_default_1769226179638_63512
Vagrant is currently configured to create VirtualBox synced folders with
the 'SharedFoldersEnableSymlinksCreate' option enabled. If the Vagrant
```

## 4- Run the ssh vagrant

```
PS D:\container> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Jan 24 03:52:11 UTC 2026

System load:  0.0               Processes:    89
Usage of /:   2.5% of 61.80GB   Users logged in:  0
Memory usage: 11%              IP address for eth0: 10.0.2.15
Swap usage:   0%

0 packages can be updated.
0 updates are security updates.
```

## 5- Perform operation inside VM

```

vagrant@vagrant:~$ df
Filesystem            1K-blocks    Used Available Use% Mounted on
udev                  473252      0   473252    0% /dev
tmpfs                 100912     5056   95856    6% /run
/dev/mapper/vagrant--vg-root 64800356 1618284 59860628 3% /
tmpfs                 504556      0   504556    0% /dev/shm
tmpfs                  5120       0    5120    0% /run/lock
tmpfs                 504556      0   504556    0% /sys/fs/cgroup
vagrant               337918972 9175036 328743936 3% /vagrant
tmpfs                 100908      0   100908    0% /run/user/1000
vagrant@vagrant:~$ free
              total        used        free      shared  buff/cache   available
Mem:           1009112        68064       456912         5056       484136       795592
Swap:           1003516           0       1003516
vagrant@vagrant:~$ exit
logout

```

- 6- Halt the vagrant

```

PS D:\container> vagrant halt
==> default: Attempting graceful shutdown of VM...

```

- 7- Destroy Vagrant

```

PS D:\container> vagrant destroy
default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...

```

## Part B: Containers using WSL (Windows)

### Step 1: Install WSL 2

```
wsl --install
```

Reboot the system after installation.

### Step 2: Install Ubuntu on WSL

```
wsl --install -d Ubuntu
```

**[INSERT SCREENSHOT: Ubuntu installation on WSL]**

### Step 3: Install Docker Engine inside WSL

```

sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo usermod -aG docker $USER

```



Logout and login again to apply group changes.

Docker Installation in WSL [INSERT SCREENSHOT: Docker installation process]

Docker Version Check

[INSERT SCREENSHOT: docker --version output]

---

## Step 4: Run Ubuntu Container with Nginx

```
docker pull ubuntu
```

```
docker run -d -p 8080:80 --name nginx-container nginx
```

Docker Pull and Run

[INSERT SCREENSHOT: Docker pull nginx and docker run command]

Container Running Status

[INSERT SCREENSHOT: docker ps showing nginx-container running]

---

## Step 5: Verify Nginx in Container

```
curl localhost:8080
```

Nginx Verification in Container

[INSERT SCREENSHOT: curl output showing Nginx welcome page from container]

---

## Resource Utilization Observation

### VM Observation Commands

```
free -h  
htop  
systemd-analyze
```

VM Resource Usage

[INSERT SCREENSHOT: free -h output in VM]

VM Boot Time Analysis

[INSERT SCREENSHOT: systemd-analyze output]

---

Container Observation Commands

```
docker stats
free -h
```

Container Resource Usage

[INSERT SCREENSHOT: docker stats output]

Host System Resource Usage

[INSERT SCREENSHOT: free -h output on host with container running]

---

Parameters to Compare and Observations

| Parameter    | Virtual Machine | Container |
|--------------|-----------------|-----------|
| Boot Time    | High            | Very Low  |
| RAM Usage    | High            | Low       |
| CPU Overhead | Higher          | Minimal   |
| Disk Usage   | Larger          | Smaller   |
| Isolation    | Strong          | Moderate  |

---

Result

The experiment demonstrates that containers are significantly more lightweight and resource-efficient compared to virtual machines, while virtual machines provide stronger isolation and full OS-level abstraction.

Virtual Machine:

- Resource overhead: High
- Isolation: Strong

Container:

- Resource overhead: Minimal
- Isolation: Good

---

## Conclusion

Virtual Machines are suitable for full OS isolation and legacy workloads, whereas Containers are ideal for microservices, rapid deployment, and efficient resource utilization.

Lab -2

## Experiment 2:- Docker Installation, Configuration, and Running Image

---

### Step 1: Pull Image

```
docker --version
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker --version  
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

### Step 2: Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d -p 8080:80 nginx  
ebc025eb75510454b9c9bec8d329e5cf212df6802e3bdc7a1c6504112489c948
```

### Step 3: Verify Running Containers

```
docker ps
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
ebc025eb7551   nginx    "/docker-entrypoint..." 46 seconds ago Up 45 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/t
```

## Step 4: Stop Container

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker stop <container id>
aditya@Aditya:/mnt/c/Users/Arundun$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
b8f593b6fa9c   ubuntu    "/bin/bash"             44 seconds ago Exited (0) 14 seconds ago
ebc025eb7551   nginx    "/docker-entrypoint..." 21 minutes ago Up 21 minutes 0.0.0.0:8080->80/tcp, [
a0cf0fa3166e   nginx    "/docker-entrypoint..." 9 hours ago    Exited (255) 22 minutes ago 0.0.0.0:8080->80/tcp, [
5a690e1e2daf   busybox   "sh"                    2 days ago     Exited (255) 2 days ago
b217a45b07e4   nginx    "/docker-entrypoint..." 2 days ago     Exited (255) 2 days ago 80/tcp
2a4009f51f94   mysql    "docker-entrypoint.s..." 8 days ago     Created
aditya@Aditya:/mnt/c/Users/Arundun$ docker stop mycontainer
mycontainer
aditya@Aditya:/mnt/c/Users/Arundun$
```

## Step 5: Remove Container

```
docker rm <container_id>
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker stop mycontainer
mycontainer
aditya@Aditya:/mnt/c/Users/Arundun$ docker rm mycontainer
mycontainer
aditya@Aditya:/mnt/c/Users/Arundun$
```

## Step 6: Remove Image

```
docker rmi nginx
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d -p 8080:80 --name my-nginx nginx
8120db9b62a203bdb077bf4127d69f137158bdd42b8cc8655a4e4c4eaf120f02
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint my-nginx (2918eb2d887d21ec33c53b45fd100d6263e31b9647b50e91d4f0995875c35a28) : Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
aditya@Aditya:/mnt/c/Users/Arundun$ docker rm my-nginx
my-nginx
```

**Result** Docker images were successfully pulled, containers executed, and lifecycle commands performed.

## Experiment 3:- Docker Deploy NGINX Using Different Base Images and Comparing Image Layers

---

## Part 1: Deploy NGINX Using Official Image (Recommended Approach)

---

### Step 1: Pull the Image

```
docker pull nginx:latest
```

```
PS C:\Users\Arundun> wsl
aditya@Aditya:/mnt/c/Users/Arundun$ docker pull nginx:latest
latest: Pulling from library/nginx
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

### Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d --name nginx-official -p 8080:80 nginx
51816bd7e7606e97c103675a427e86a43dcc60955e903a57cf31f62e0aa9f198
aditya@Aditya:/mnt/c/Users/Arundun$
```

### Step 3: Verify

```
curl http://localhost:8080
aditya@Aditya:/mnt/c/Users/Arundun$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>
```

You should see the **NGINX welcome page**.

### Key Observations

```
docker images nginx
aditya@Aditya:/mnt/c/Users/Arundun$ docker images nginx
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    341bf0f3ce6c   2 weeks ago   240MB
```

- Image is **pre-optimized**
- Minimal configuration required
- Uses Debian-based OS internally

## Part 2: Custom NGINX Using Ubuntu Base Image

### Step 1: Create Dockerfile

```

dockerfile X
C: > Users > Arundun > Desktop > docker > dockerfile
1  FROM ubuntu:22.04
2
3  RUN apt-get update && \
4      apt-get install -y nginx && \
5      apt-get clean && \
6      rm -rf /var/lib/apt/lists/*
7
8  EXPOSE 80
9
10 CMD ["nginx", "-g", "daemon off;"]

```

### Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

```

^Caditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker build -t nginx-ubuntu .
[+] Building 283.7s (7/7) FINISHED                                docker:default
=> [internal] load build definition from dockerfile                0.1s
=> => transferring dockerfile: 226B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04    6.5s
=> [auth] library/ubuntu:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c52 231.4s
=> resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a 0.1s
=> sha256:blcha2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed 29.54MB / 29.54MB 230.0s
=> extracting sha256:blcha2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed 1.2s
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lis 41.7s
=> exporting to image                                              3.7s
=> exporting layers                                                2.7s
=> exporting manifest sha256:6d0553f600461c83368ae50961ab6e23cdc383dbe0978783686b0e36a3c97347 0.0s
=> exporting config sha256:bfc970f6e4020fclfa49ff9465f91533cd0c89c5fa2f0abf1bed5280fcb89cb9 0.0s
=> exporting attestation manifest sha256:d76230a6d3fc64f315546ae69447f1b1416f918494709bd51f609dcb7299db0b 0.1s
=> exporting manifest list sha256:c85599b0ef3d4a97100bd1f5498983678b25f47737b7bbd4e9fac067c94b0f09 0.0s
=> naming to docker.io/library/nginx-ubuntu:latest                0.0s
=> unpacking to docker.io/library/nginx-ubuntu:latest             0.8s

```

### Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
4e612c1f28fbd53b27ed4702f23c06686f2d0987b7b5c342a9faaf7aafd677d6
```

### Observations

```
docker images nginx-ubuntu
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker images nginx-ubuntu
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx-ubuntu latest c85599b0ef3d 9 minutes ago 199MB
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$
```

- Much **larger image size**
- More layers
- Full OS utilities available

---

## Part 3: Custom NGINX Using Alpine Base Image

---

### Step 1: Create Dockerfile

```
new > 🐳 dockerfile
1 FROM alpine:latest
2
3 RUN apk add --no-cache nginx
4
5 EXPOSE 80
6
7 CMD ["nginx", "-g", "daemon off;"]
```

### Step 2: Build Image

```
docker build -t nginx-alpine .
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker build -t nginx-alpine .
[+] Building 6.1s (7/7) FINISHED                                docker:default
=> [internal] load build definition from dockerfile             0.1s
=> => transferring dockerfile: 226B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04 5.5s
=> [auth] library/ubuntu:pull token for registry-1.docker.io   0.0s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 28                                   0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a 0.1s
=> => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a 0.0s
=> CACHED [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/a 0.0s
=> exporting to image                                           0.2s
=> => exporting layers                                           0.0s
=> => exporting manifest sha256:6d0553f600461c83368ae50961ab6e23cdc383dbe0978783686b0e36a3c97347 0.0s
=> => exporting config sha256:bfc970f6e4020fclfa49ff9465f91533cd0c89c5fa2f0abf1bed5280fcb89cb9 0.0s
=> => exporting attestation manifest sha256:974594a387191fdb3b52e58d7ea3fb52f471e5d6f485d25e7faf6ccc26dd9dd9 0.1s
=> => exporting manifest list sha256:e56d6c3ab8e2b65695b8eec921ef3cbb2e4668adafce369e88bb9bce40d1568c 0.0s
=> => naming to docker.io/library/nginx-alpine:latest          0.0s
=> => unpacking to docker.io/library/nginx-alpine:latest       0.0s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$
```

### Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
fbd78c92bba2931f392a71aeffa651014a21d6e588d6adc7del43f1294f267a
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$
```

### Observations

```
docker images nginx-alpine
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker images nginx-alpine
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx-alpine latest e56d6c3ab8e2 18 minutes ago 199MB
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$
```

- Extremely **small image**
- Fewer packages
- Faster pull and startup time

## Part 4: Image Size and Layer Comparison

### Compare Sizes

```
docker images | grep nginx
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker images | grep nginx
nginx-alpine latest e56d6c3ab8e2 18 minutes ago 199MB
nginx-ubuntu latest c85599b0ef3d 18 minutes ago 199MB
nginx latest 341bf0f3ce6c 2 weeks ago 240MB
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$
```

### Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```



```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
341bf0f3ce6c   2 weeks ago     CMD ["nginx" "-g" "daemon off;"] 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     STOPIGNAL SIGQUIT   0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     EXPOSE map[80/tcp:{}] 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENTRYPOINT ["/docker-entrypoint.sh"] 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     COPY docker-entrypoint.sh / # buildkit 8.19kB    buildkit.dockerfile.v0
<missing>      2 weeks ago     RUN /bin/sh -c set -x && groupadd --syst... 86.6MB    buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV DYNPKG_RELEASE=1-trixie 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV PKG_RELEASE=1-trixie 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV ACME_VERSION=0.3.1 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_RELEASE=1-trixie 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NJS_VERSION=0.9.5 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     ENV NGINX_VERSION=1.29.5 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      3 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB    debuerreotype 0.17
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
c85599b0ef3d   19 minutes ago  CMD ["nginx" "-g" "daemon off;"] 0B        buildkit.dockerfile.v0
<missing>      19 minutes ago  EXPOSE [80/tcp] 0B        buildkit.dockerfile.v0
<missing>      19 minutes ago  RUN /bin/sh -c apt-get update && apt-get... 58.6MB    buildkit.dockerfile.v0
<missing>      13 days ago     /bin/sh -c #(nop) CMD ["/bin/bash"] 0B
<missing>      13 days ago     /bin/sh -c #(nop) ADD file:52c0e467fa2e92f10... 87.5MB
<missing>      13 days ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B
<missing>      13 days ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B
<missing>      13 days ago     /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH 0B
<missing>      13 days ago     /bin/sh -c #(nop) ARG RELEASE 0B
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
e56d6c3ab8e2   19 minutes ago  CMD ["nginx" "-g" "daemon off;"] 0B        buildkit.dockerfile.v0
<missing>      19 minutes ago  EXPOSE [80/tcp] 0B        buildkit.dockerfile.v0
```

## Observations:

- Ubuntu has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

## Part 5: Functional Tasks Using NGINX

### Task 1: Serve Custom HTML Page

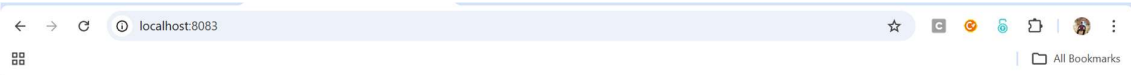
```
mkdir html
echo "<h1>Aditya Sharma - 500122015</h1>" > html/index.html
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ mkdir html
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$ echo "<h1>Aditya Sharma - 500122015</h1>" > html/index.html
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/docker$
```

Run:

```
docker run -d \
-p 8083:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx
```

```
aditya@Aditya: /mnt/c/Users/Arundun/Desktop/docker$ docker run -d \
-p 8083:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx
34fbf6f5345c5ca6650fd514054ac98af3495b0bc49e9c6c1cc0765caa518e45
aditya@Aditya: /mnt/c/Users/Arundun/Desktop/docker$ |
```



Aditya Sharma - 500122015

Task 2: Reverse Proxy (Conceptual)

NGINX can:

- Forward traffic to backend services
- Load balance multiple containers
- Terminate SSL

Example use cases:

- Frontend for microservices
- API gateway
- Static file server



Part 6: Comparison Summary



Image Comparison Table

| Feature      | Official NGINX | Ubuntu + NGINX |
|--------------|----------------|----------------|
| Image Size   | Medium         | Large          |
| Ease of Use  | Very Easy      | Medium         |
| Startup Time | Fast           | Slow           |

| Feature          | Official NGINX | Ubuntu + NGINX |
|------------------|----------------|----------------|
| Debugging Tools  | Limited        | Excellent      |
| Security Surface | Medium         | Large          |
| Production Ready | Yes            | Rarely         |

---

## Part 7: When to Use What

---

### Use Official NGINX Image When:

- Production deployment
- Standard web hosting
- Reverse proxy / load balancer

### Use Ubuntu-Based Image When:

- Learning Linux + NGINX internals
- Heavy debugging needed
- Custom system-level dependencies

### Use Alpine-Based Image When:

- Microservices
- CI/CD pipelines
- Cloud and Kubernetes workloads

## Experiment 5:

### Docker - Volumes, Environment Variables,

### Monitoring & Networks

## Part 1: Docker Volumes - Persistent Data Storage

### Understanding Data Persistence

The Problem: Container Data is Ephemeral

Create a container that writes data

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
b40150c1c271: Pull complete
92842f25412d: Download complete
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
Status: Downloaded newer image for ubuntu:latest
root@402aab63ac20:/#
```

Inside container:

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -it --name test-container ubuntu /bin/bash
root@769cf42969ba:/# echo "Hello World" > /data/message.txt
bash: /data/message.txt: No such file or directory
root@769cf42969ba:/# mkdir /data
echo "Hello World" > /data/message.txt
root@769cf42969ba:/# cat /data/message.txt
Hello World
root@769cf42969ba:/# exit
exit
aditya@Aditya:/mnt/c/Users/Arundun$ |
```

docker start test-container

docker exec test-container cat /data/message.txt

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker start test-container
test-container
aditya@Aditya:/mnt/c/Users/Arundun$ docker exec test-container cat /data/message.txt
Hello World
aditya@Aditya:/mnt/c/Users/Arundun$ |
```

### Volume Types

#### 1. Anonymous Volumes

Create anonymous volume (auto-generated name)

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d -v /app/data --name web1 nginx
a9417602c745920b1f165fc38c48c43606c3dad0ec3ce8c07348317b8dcc674
aditya@Aditya:/mnt/c/Users/Arundun$ |
```

Check volume

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume ls
DRIVER      VOLUME NAME
local       2dee95a6a5486ec4c7474bd527d5434df3a1c0a39a010f4c1c332c051b8b1472
aditya@Aditya:/mnt/c/Users/Arundun$
```

## Inspect container to see volume mount

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker inspect web1 | grep -A 5 Mounts
  "Mounts": [
    {
      "Type": "volume",
      "Name": "2dee95a6a5486ec4c7474bd527d5434df3a1c0a39a010f4c1c332c051b8b1472",
      "Source": "/var/lib/docker/volumes/2dee95a6a5486ec4c7474bd527d5434df3a1c0a39a010f4c1c332c051b8b1472/_data",
      "Destination": "/app/data",
      "Driver": "local"
    }
  ]
aditya@Aditya:/mnt/c/Users/Arundun$
```

## Named Volumes

### Create named volume

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume create mydata
mydata
aditya@Aditya:/mnt/c/Users/Arundun$
```

### Use named volume

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d -v mydata:/app/data --name web2 nginx
bcb91723ef7cc4c982db4d5a8d3dd5f31461735aa1d25f68b9f7d80145ddcc7b
aditya@Aditya:/mnt/c/Users/Arundun$
```

### List volumes

#### Inspect volume

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume ls
DRIVER      VOLUME NAME
local       2dee95a6a5486ec4c7474bd527d5434df3a1c0a39a010f4c1c332c051b8b1472
local       mydata
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume inspect mydata
[
  {
    "CreatedAt": "2026-04-17T17:40:05Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
```

## Bind Mounts (Host Directory)

### Create directory on host

### Mount host directory to container

```
aditya@Aditya:/mnt/c/Users/Arundun$ mkdir ~/myapp-data
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d -v ~/myapp-data:/app/data --name web3 nginx
7b025b661c12cad202ebe65472b3d1c9311e535368f1d8c74277575fd1d2f1ad
aditya@Aditya:/mnt/c/Users/Arundun$
```

### Add file on host

Check in container

```
aditya@Aditya:/mnt/c/Users/Arundun$ echo "From Host" > ~/myapp-data/host-file.txt
aditya@Aditya:/mnt/c/Users/Arundun$ docker exec web3 cat /app/data/host-file.txt
From Host
aditya@Aditya:/mnt/c/Users/Arundun$ |
```

## Practical Volume Examples

Example 1: Database with Persistent Storage

MySQL with named volume

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
26e72ef22e05: Pull complete
c2ccadc91b4f: Pull complete
bb5107df7baa: Pull complete
496eb3815f35: Pull complete
a966b78a59e9: Pull complete
7fc827b16c7b: Pull complete
c350fcbc37b3: Pull complete
a77ecc122bf4: Pull complete
b6126bf990b9: Pull complete
59a1a4fb3367: Pull complete
e69c68c7dd21: Pull complete
0faffc9d5b78: Download complete
c0f0e180252c: Download complete
Digest: sha256:4af1f8815716546f5b12410f7621f37f93db8dd11a184706ef59111930b8c2ff
Status: Downloaded newer image for mysql:8.0
45ef0df4009d31d4b863d467b21c709106119cfa62d074d1648c5a995162de15
aditya@Aditya:/mnt/c/Users/Arundun$
```

Check data persists

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker stop mysql-db
mysql-db
aditya@Aditya:/mnt/c/Users/Arundun$ docker rm mysql-db
mysql-db
```

New container with same volume

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
73f1a659a13583f0077679352d237d6cd1ad1b59aed2dc54699c5bf0ed11eb4c
aditya@Aditya:/mnt/c/Users/Arundun$ |
```

Data is preserved!

## Volume Management Commands

List all volumes

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume ls
DRIVER      VOLUME NAME
local       2dee95a6a5486ec4c7474bd527d5434df3a1c0a39a010f4c1c332c051b8b1472
local       mydata
local       mysql-data
```

Create a volume

Inspect volume details

Remove unused volumes

```
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume create app-volume
app-volume
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-18T15:56:53Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
aditya@Aditya:/mnt/c/Users/Arundun$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
aditya@Aditya:/mnt/c/Users/Arundun$ |
```

## Part 2: Environment Variables

### Setting Environment Variables

Method 1: Using -e flag



```

PS C:\Users\Arundun\Desktop\Containers\Container\LAB\lab5> wsl
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ docker run -d \
--name appl \
-e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
-e DEBUG="true" \
-p 3000:3000 \
my-node-app
Unable to find image 'my-node-app:latest' locally
docker: Error response from daemon: pull access denied for my-node-app, repository does not exist or may require 'docker login'

Run 'docker run --help' for more information
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ docker build -t my-node-app .
[+] Building 19.9s (5/8)
=> [internal] load build definition from dockerfile                                docker:default
=> => transferring dockerfile: 118B                                              0.1s
=> [internal] load metadata for docker.io/library/node:18                      4.8s
=> [auth] library/node:pull token for registry-1.docker.io                    0.0s
=> [internal] load .dockerignore                                                0.1s
=> => transferring context: 2B                                                  0.0s
=> [1/3] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ecd224d96c693a8a4d943498556716d3783 14.8s
=> => resolve docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ecd224d96c693a8a4d943498556716d3783 0.0s
=> => sha256:cda7f44f2bddcc4bb7514474824b3f3705de08ddb6355a33be5ac7808e5b7125 3.32kB / 3.32kB 0.7s
=> => sha256:461077a72fb7fe40d34a37d6a1958c4d16772d0dd77f572ec50a1fdc41a3754d 446B / 446B 1.1s
=> => sha256:e23f099911d692f62b851cf49a1e93294288a115f5cd2d014180e4d3684d34ab 30.41MB / 211.36MB 14.5s
=> => sha256:3697be50c98b9d071df4637e1d3491d00e7b9f3a732768c876d82309b3c5a145 1.25MB / 1.25MB 1.9s
=> => sha256:c6b30c3f16966552af10ac00521f60355b1fcfd46ac1c20b1038587e28583ce7 31.46MB / 45.68MB 13.8s
=> [internal] load build context
=> => transferring context: 491B                                              0.1s

```

## Single variable

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ docker run -d \
--name appl \
-e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
-e DEBUG="true" \
-p 3000:3000 \
my-node-app
25fcff4a38b9ef989d3650aeeb1ac18e66e9fe2b2ed0f92c36eb870ab34d5d5
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ |

```

## Multiple variables

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ docker run -d \
-e VAR1=value1 \
-e VAR2=value2 \
-e VAR3=value3 \
my-node-app
25842c720b2f648600e0f4d91754553c61554abb0d264f7e8e81b9f91d0d52d5
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ |

```

## Method 2: Using --env-file

### Use env file

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ docker run -d \
--env-file .env \
--name app2 \
my-node-app
c6076622f12ae5d4047b059729c216adb47b2ca9bdd98c967fe5b97ea7f3e5d
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ |

```

### Use multiple env files

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ docker run -d --env-file .env --env-file .env.secrets
my-node-app
2edd51a188b5ee47c78975573c69aef316519d5210b273b4dfadbb581c3bad9
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5$ |

```



# Environment Variables in Applications

## Python Flask Example

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ docker build -t flask-app .
[+] Building 17.8s (10/10) FINISHED
=> [internal] load build definition from dockerfile                                docker:default
=> => transferring dockerfile: 376B                                              0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim                1.5s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 28                                                    0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b 0.0s
=> [internal] load build context                                                  0.1s
=> => transferring context: 78B                                                  0.1s
=> CACHED [2/5] WORKDIR /app                                                    0.0s
=> [3/5] COPY requirements.txt .                                                 0.0s
=> [4/5] RUN pip install -r requirements.txt                                    14.1s
=> [5/5] COPY app.py .                                                         0.1s
=> exporting to image                                                            1.5s
=> => exporting layers                                                            0.9s
=> => exporting manifest sha256:884662031d20dd85a3ef3dd70c4844660bb1094c629c3b7f1690eacfbcb895896          0.0s
=> => exporting config sha256:46e4fbf3e85e7297e7097cb3e5e4b5b7150d2854da0e1c067ab2cb91db08e9ba          0.0s
=> => exporting attestation manifest sha256:48bd5cc4c9b2f924eef9dd9b0d2fc00e4ac9083850f77236c28313166ca09fb2    0.1s
=> => exporting manifest list sha256:e72bc10237be6d67e29a852d660ceef3893b622b05fa336f65bb77fb65b76b82        0.0s
=> => naming to docker.io/library/flask-app:latest                             0.0s
=> => unpacking to docker.io/library/flask-app:latest                           0.4s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$
```

## Lab 3: Test Environment Variables

Run with custom env vars

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ docker run -d \
--name flask-app \
-p 5000:5000 \
-e DATABASE_HOST="prod-db.example.com" \
-e DEBUG="true" \
-e PORT="8080" \
flask-app
d067045b163b94a79bfaca06a732ef193400be4b8f816b3ad6bfcc9961c5f819
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ |
```

Check environment in running container

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ docker exec flask-app env
docker exec flask-app printenv DATABASE_HOST
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=d067045b163b
DATABASE_HOST=prod-db.example.com
DEBUG=true
PORT=8080
LANG=C.UTF-8
GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568
PYTHON_VERSION=3.9.25
PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834
PYTHONUNBUFFERED=1
PYTHONDONTWRITEBYTECODE=1
HOME=/root
prod-db.example.com
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ |
```

Test the endpoint

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ curl http://localhost:5000/config
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": false
}
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab5/2nd$ |
```

## Experiment 6 A

### Title: Comparison of Docker Run and Docker Compose

## PART A – THEORY

### 1. Objective

To understand the relationship between `docker run` and Docker Compose, and to compare their configuration syntax and use cases.

### 2. Background Theory

#### 2.1 Docker Run (Imperative Approach)

The `docker run` command is used to create and start a container from an image. It requires explicit flags for:

- Port mapping (`-p`)
- Volume mounting (`-v`)
- Environment variables (`-e`)
- Network configuration (`--network`)
- Restart policies (`--restart`)
- Resource limits (`--memory`, `--cpus`)
- Container name (`--name`)

This approach is **imperative**, meaning you specify step-by-step instructions.

Example:

```
docker run -d \
  --name my-nginx \
  -p 8080:80 \
  -v ./html:/usr/share/nginx/html \
  -e NGINX_HOST=localhost \
  --restart unless-stopped \
  nginx:alpine
```

2/31

#### 2.2 Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format.

Instead of multiple `docker run` commands, a single command is used:

```
docker compose up -d
```

Compose is **declarative**, meaning you define the desired state of the application.

Equivalent Compose file:

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
      NGINX_HOST: localhost
```

```
restart: unless-stopped
```

3/31

### 3. Mapping: Docker Run vs Docker Compose

#### Docker Run Flag

```
-p 8080:80
-v host:container
-e KEY=value
--name
--network
--restart
--memory
--cpus
-d
```

#### Docker Compose Equivalent

```
ports:
volumes:
environment:
container_name:
networks:
restart:
deploy.resources.limits.memory
deploy.resources.limits.cpus
docker compose up -d
```

4/31

### 4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling

Example:

```
docker compose up --scale web=3
```

5/31

## PART B – PRACTICAL TASK

### Task 1: Single Container Comparison

#### Step 1: Run Nginx Using Docker Run

Execute:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$ docker run -d \
--name lab-nginx \
-p 8081:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine
1d089ed22aa61837d5020b0a8871be81ab5a2c3eddd0bfa64a8672bf8c2eb0be
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$
```

Verify:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
1d089ed22aa6   nginx:alpine   "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
lab-nginx
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$
```

Access:

<http://localhost:8081>



Stop and remove container:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$ docker stop lab-nginx
lab-nginx
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$ docker rm lab-nginx
lab-nginx
aditya@Aditya:/mnt/c/Users/Arundun/Desktop$
```

## Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml`:

```
Container > LAB > lab6 > docker-compose.yml
1  version: '3.8'
2
3  services:
4    nginx:
5      image: nginx:alpine
6      container_name: lab-nginx
7      ports:
8        - "8081:80"
9      volumes:
10       - ./html:/usr/share/nginx/html
11
```

Run:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker compose up -d
WARN[0000] /mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
[+] up 2/2
✔ Network lab6_default Created 0.0s
✔ Container lab-nginx Created 0.2s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$
```

Verify:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker compose ps
WARN[0000] /mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME        IMAGE               COMMAND             SERVICE    CREATED         STATUS         PORTS
lab-nginx   nginx:alpine        "/docker-entrypoint..." nginx       20 seconds ago  Up 19 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ |
```

Stop:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker compose down
WARN[0000] /mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container lab-nginx   Removed                                0.4s
✔Network lab6_default Removed                                0.3s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ |
```

## Task 2: Multi-Container Application

### Objective:

Deploy WordPress with MySQL using:

1. Docker Run (manual way)
2. Docker Compose (structured way)

8/31

### A. Using Docker Run

1. Create network:

1. Run MySQL:

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker network create wp-net
6927cc706ef729889030661211a8da7d1258f1b2bb7ea6edcf98e563ebbe7acb
```

1. Run WordPress:

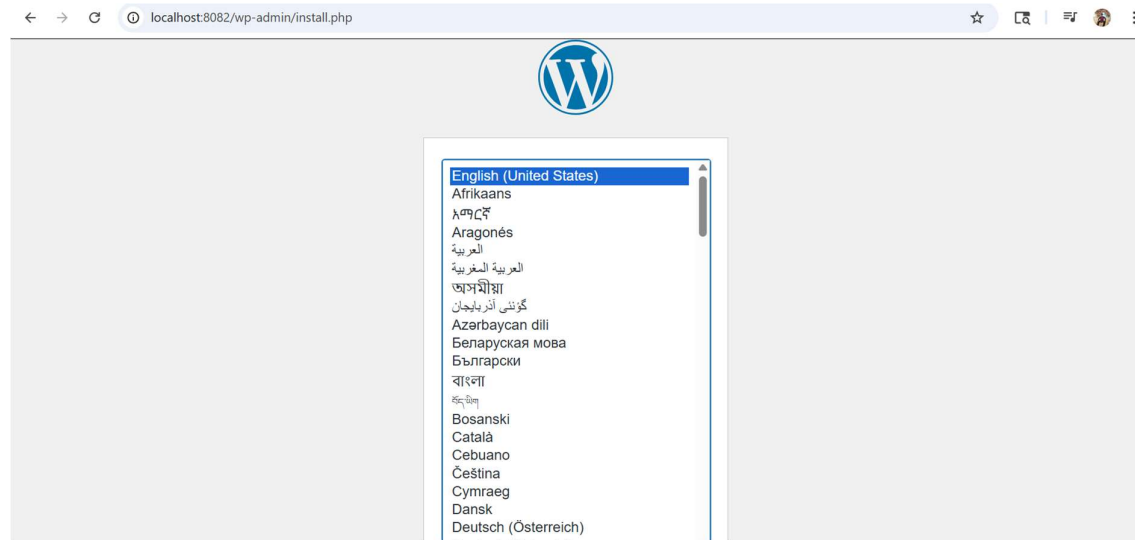
```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret wordpress:latest
ffebd2d73a52e34bcd302054f7434bc65af5b9d8e919f3b90bdb714d5275d46c
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ |
```

Test:

<http://localhost:8082>

9/31



## B. Using Docker Compose

Create `docker-compose.yml`:

```
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```

version: '3.8'

Run:

```
docker compose up -d
```

Stop:

```
docker compose down -v
```

10/31

## PART C – CONVERSION & BUILD-BASED TASKS

### Task 3: Convert Docker Run to Docker Compose

#### Problem 1: Basic Web Application

##### Given Docker Run Command:

```
docker run -d \
  --name webapp \
  -p 5000:5000 \
  -e APP_ENV=production \
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine
```

11/31

##### Student Task:

1. Write an equivalent `docker-compose.yml`
2. Ensure:
  - o Same container name
  - o Same port mapping
  - o Same environment variables
  - o Same restart policy
3. Run using:

```
docker compose up -d
```

1. Verify using:

```
docker compose ps
```

12/31

#### Problem 2: Volume + Network Configuration

##### Given Docker Run Commands:

```
docker network create app-net
docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
```



```
postgres:15

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker network create app-net
d8f7ee684a0d9b2a3931a5e45b7f411f2dabb240b203334a1fe417139b0d31cb
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker run -d \
--name postgres-db \
--network app-net \
-e POSTGRES_USER=admin \
-e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data \
postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
a8a0725a32a6: Pull complete
2c38b75ffca2: Pull complete
71a32c7571f4: Pull complete
00f33d580dc3: Pull complete
9328c154beae: Pull complete
f7049a93e93f: Pull complete
af2a6e5f285c: Pull complete
af0c3ea2fe5e: Pull complete
b4765ac2e7e6: Pull complete
5c152148754e: Pull complete
8ecd190da6: Pull complete
c45b346571fc: Pull complete
5ca81a59b0a4: Pull complete
b9015a4f6b87: Download complete
cebcb07837b: Download complete
Digest: sha256:3e43515057e113ee741fca2f621f15300be34a6d4a8dfcf2e20651288c8272f3
Status: Downloaded newer image for postgres:15

docker run -d \
--name backend \
--network app-net \
-p 8000:8000 \
-e DB_HOST=postgres-db \
-e DB_USER=admin \
-e DB_PASS=secret \
python:3.11-slim
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker run -d \
--name backend \
--network app-net \
-p 8000:8000 \
-e DB_HOST=postgres-db \
-e DB_USER=admin \
-e DB_PASS=secret \
python:3.11-slim
Unable to find image 'python:3.11-slim' locally
3.11-slim: Pulling from library/python
0816ee351b60: Pull complete
d98f36ba2159: Pull complete
37ffa6577b04: Pull complete
6f70bfdbc954: Download complete
79505fc62d4b: Download complete
Digest: sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
Status: Downloaded newer image for python:3.11-slim
54ae67a4775d9368e29a8bb6558a0ad5aeb414880d4f018054f783e8e0b60f1e
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$
```

13/31

### Student Task:

1. Create a single `docker-compose.yml` file that:
  - o Defines both services
  - o Creates named volume `pgdata`
  - o Creates custom network `app-net`
  - o Uses `depends_on`
2. Bring up services using one command.
3. Stop and remove everything properly.

14/31

### Task 4: Resource Limits Conversion Given Docker Run Command:

```
docker run -d \
```

```
--name limited-app \
-p 9000:9000 \
--memory="256m" \
--cpus="0.5" \
--restart always \
nginx:alpine
```

15/31

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab6$ docker run -d \
--name limited-app \
-p 9000:9000 \
--memory="256m" \
--cpus="0.5" \
--restart always \
nginx:alpine
15/31
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
612c0c1df4c5: Downloading [=====] 16.78MB/20.25MB
aee4e54b3865: Pull complete
781ff50d2644: Pull complete
6a0ac1617861: Pull complete
453da7dbc73e: Pull complete
4a8b0b2a5b19: Pull complete
82736a35d0e7: Pull complete
583599bb7d38: Pull complete
e8fc446e336c: Download complete
6192e1e6a438: Download complete
```

16/31

## PART D – USING DOCKERFILE INSTEAD OF STANDARD IMAGE

Now increase difficulty.

Instead of using prebuilt images like:

image: `node:18-alpine`

Students must:

- Create their own Dockerfile
- Build image using Compose
- Run container from that build

17/31

### Explanation of Key Sections

#### services

Defines containers:

- `db` → MySQL database
- `wordpress` → application

#### depends\_on

- Ensures DB starts before WordPress

#### environment

- Used to configure DB credentials and connection

### volumes

- Persist data even if containers are deleted

### ports

- Exposes WordPress on:

`http://localhost:8080`

## Step 3: Start Application

`docker-compose up -d`

What happens:

- Images are pulled
- Network is created
- Containers are started
- DNS-based service discovery enabled

## Step 4: Verify Containers

`docker ps`

Expected:

- `wordpressapp`
- `wordpressdb`

## Step 5: Access WordPress

Open browser:

`http://localhost:8080`

- Complete WordPress setup
- Enter site title, admin user, password

## Step 6: Check Volumes

`docker volume ls`

- `dbdata` → *database persistence*
- `wpdata` → WordPress files

## Step 7: Stop Application

`docker-compose down`

- Containers removed
- Volumes remain intact

28/31

## 5. Scaling in Docker Compose

### Method 1: Scale WordPress Containers

`docker-compose up --scale wordpress=3`

Result:

- 3 WordPress containers running

### Problem:

- All try to use same port (8080)
- No load balancing

### Solution: Use Reverse Proxy (Nginx)

Add another service:

```
nginx:
  image: nginx:latest
  ports:
    - "8080:80"
```

Then configure load balancing manually.

### Limitations of Compose Scaling

- No built-in load balancing
- No auto-healing
- Single host only
- Not production-ready for scaling

29/31

## 6. Running Same Setup with Docker Swarm

### Step 1: Initialize Swarm

```
docker swarm init
```

### Step 2: Deploy Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

### Step 3: Scale Service

```
docker service scale wpstack_wordpress=3
```

### What Changes in Swarm?

| Feature         | Docker Compose | Docker Swarm       |
|-----------------|----------------|--------------------|
| Scope           | Single host    | Multi-node cluster |
| Scaling         | Manual         | Built-in           |
| Load balancing  | No             | Yes (internal LB)  |
| Self-healing    | No             | Yes                |
| Rolling updates | No             | Yes                |
| Networking      | Basic          | Overlay network    |

## 7. Benefits of Docker Swarm

- Built-in load balancing
- Automatic container restart (self-healing)
- Horizontal scaling across nodes
- Rolling updates without downtime
- Service abstraction (not individual containers)

## 8. Challenges / Limitations of Swarm

- Less popular than Kubernetes
- Limited ecosystem
- Less flexible scheduling
- Fewer enterprise features

## 9. Key Learning Outcomes

- Multi-container apps require orchestration
- Docker Compose is ideal for:
  - Development
  - Testing
  - Learning
- Docker Swarm is useful for:
  - Simple production clusters
  - Easy scaling without Kubernetes complexity

30/31

## 10. Conclusion

This experiment demonstrated:

- How to deploy **WordPress + MySQL** using Docker Compose
- How containers communicate using internal networking
- Importance of **volumes for persistence**
- Scaling limitations of Compose
- Advantages of using **Docker Swarm for production-ready deployments**

31/31

## Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

### 1. Aim

To design and implement a complete CI/CD pipeline using **Jenkins**, integrating source code from **GitHub**, and building & pushing Docker images to **Docker Hub**.

### 2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

### 3. Theory

#### What is Jenkins?

Jenkins is a **web-based GUI automation server** used to:

- Build applications
- Test code
- Deploy software

It provides:

- Dashboard (browser-based UI)
- Plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using `Jenkinsfile`

## What is CI/CD?

- **Continuous Integration (CI):**

Code is automatically built and tested after each commit

- **Continuous Deployment (CD):**

Built artifacts (Docker images) are automatically delivered/deployed

## Workflow Overview

Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

## 4. Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

## 5. Part A: GitHub Repository Setup (Source Code + Build Definition)

### 5.1 Create Repository

Create a repository on GitHub:

`my-app`

### 5.2 Project Structure

```
my-app/
├── app.py
├── requirements.txt
├── Dockerfile
└── Jenkinsfile
```

### 5.3 Application Code

`app.py`

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"
    #return "Hello from CI/CD Pipeline!, my sapid is 123456"

app.run(host="0.0.0.0", port=80)
```

`requirements.txt`

flask

### 5.4 Dockerfile (Build Process)

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt
```

EXPOSE 80  
CMD ["python", "app.py"]

## Build Process Explanation

1. Source code pushed to GitHub
2. Jenkins pulls code
3. Dockerfile:
  - Creates environment
  - Installs dependencies
  - Packages app
4. Output → Docker Image

## 5.5 Jenkinsfile (Pipeline Definition in GitHub)

```
1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_NAME = "adibhai/myapp"
6   }
7
8   stages {
9     stage('Clone Source') {
10      steps {
11        git 'https://github.com/Aditya12155/my-app'
12      }
13    }
14
15    stage('Build Docker Image') {
16      steps {
17        sh 'docker build -t $IMAGE_NAME:latest .'
18      }
19    }
20
21    stage('Login to Docker Hub') {
22      steps {
23        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
24          sh 'echo $DOCKER_TOKEN | docker login -u adibhai --docker-pat_n_MtWdJEB5gJRBUZ_u-TqW7814'
25        }
26      }
27    }
28
29    stage('Push to Docker Hub') {
30      steps {
31        sh 'docker push $IMAGE_NAME:latest'
32      }
33    }
34  }
35 }
36
37
```

2022

## 6. Part B: Jenkins Setup using Docker (Persistent Configuration)

### 6.1 Create Docker Compose File

```
1 docker-compose.yml
2
3 version: '3.8'
4
5 services:
6   jenkins:
7     image: jenkins/jenkins:lts
8     container_name: jenkins
9     restart: always
10    ports:
11      - "8080:8080"
12      - "50000:50000"
13    volumes:
14      - jenkins_home:/var/jenkins_home
15      - /var/run/docker.sock:/var/run/docker.sock
16    user: root
17
18 volumes:
19   jenkins_home:
```

### 6.2 Start Jenkins

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/my-app$ docker-compose up -d
WARN[0000] /mnt/c/Users/Arundun/Desktop/my-app/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 14/14
[+] up 16/16kins/jenkins:lts Pulled 71.4s
✓Image jenkins/jenkins:lts Pulled 71.4s
✓e714309bcbf67 Pull complete 0.8s
✓32ad68c1bba8 Pull complete 1.0s
✓6e001067f512 Pull complete 1.2s
✓b850d1c40542 Pull complete 67.4s
✓1aac2fe67fcc Pull complete 67.4s
✓f015f18f3c7a Pull complete 58.5s
✓0454b3406328 Pull complete 58.6s
✓6e428cebe944 Pull complete 60.4s
✓beb35d8d9493 Pull complete 58.2s
✓32ebd389acde Pull complete 1.3s
✓a7730063fcfe Pull complete 55.0s
✓a576649e9376 Pull complete 58.4s
✓e925eaf895df Download complete 0.0s
✓Network my-app_default Created 0.1s
✓Container jenkins Created 0.6s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/my-app$

```

Access:

<http://localhost:8080>

Getting Started

## Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

### 6.3 Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/my-app$ docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
82f247ddd0a54e188bbe7d318e463686
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/my-app$

```

### 6.4 Initial Setup

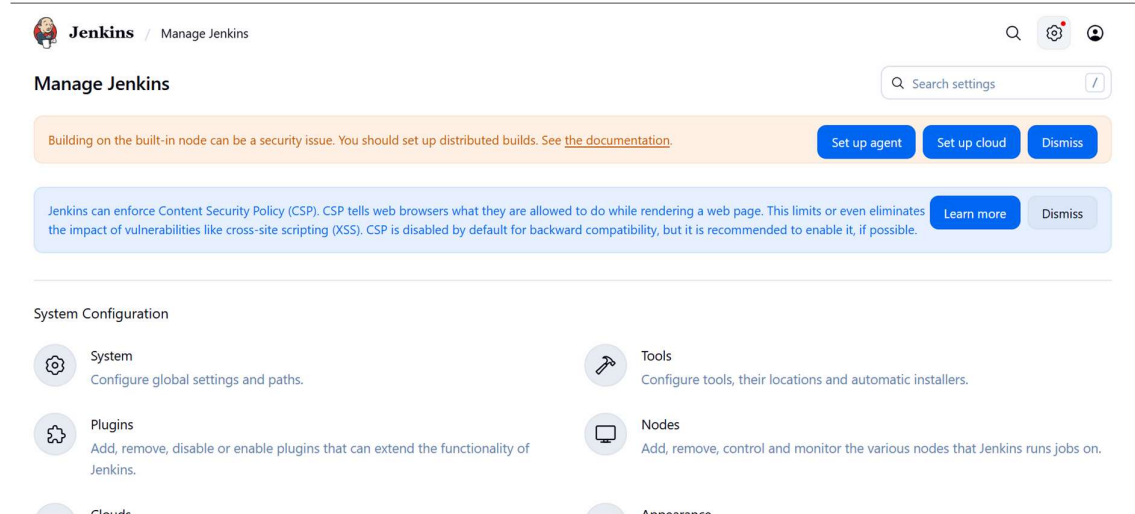
- Install suggested plugins
- Create admin user

3/12

## 7. Part C: Jenkins Configuration



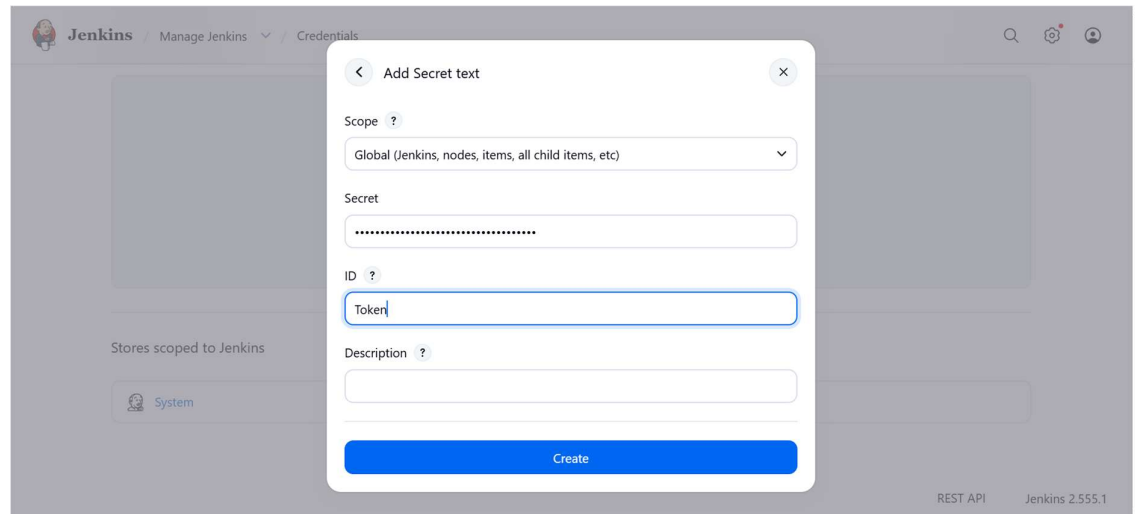
## 7.1 Add Docker Hub Credentials



Path:

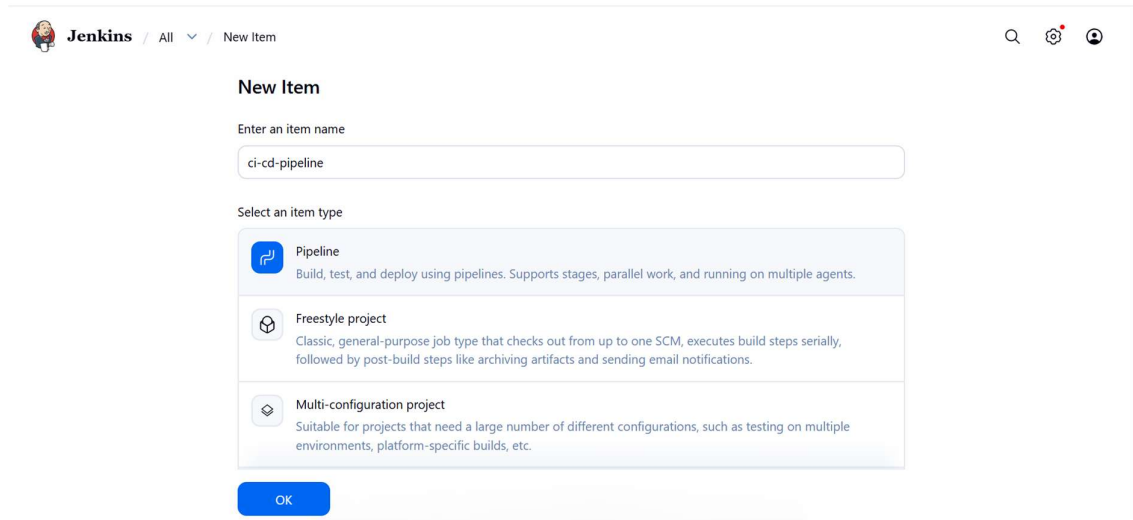
Manage Jenkins → Credentials → Add Credentials

- Type: Secret Text
- ID: dockerhub-token
- Value: Docker Hub Access Token



## 7.2 Create Pipeline Job

1. New Item → Pipeline
2. Name: ci-cd-pipeline



**Jenkins** / All / New Item

Enter an item name

ci-cd-pipeline

Select an item type

- Pipeline**  
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project  
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project  
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

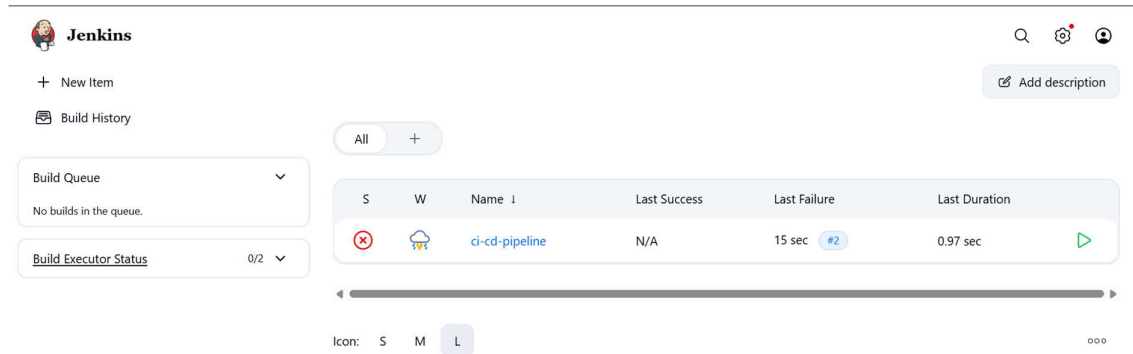
OK

## Configure:

Pipeline script from SCM

- **SCM:** Git
- **Repo URL:** your GitHub repo
- **Script Path:** Jenkinsfile

4/12



**Jenkins**

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

0/2

All +

| S | W | Name           | Last Success | Last Failure | Last Duration |
|---|---|----------------|--------------|--------------|---------------|
| ✖ | ☁ | ci-cd-pipeline | N/A          | 15 sec #2    | 0.97 sec      |

Icon: S M L

## 8. Part D: GitHub Webhook Integration

### 8.1 Configure Webhook

In GitHub:

Settings → Webhooks → Add Webhook

Payload URL:

http://<your-server-ip>:8080/github-webhook/

```
ngrok
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             adityaddn07@gmail.com (Plan: Free)
Version             3.38.0
Region              India (in)
Latency              32ms
Web Interface        http://127.0.0.1:4040
Forwarding           https://engraved-provoking-slimness.ngrok-free.dev -> http://localhost:8080

Connections          ttl    opn    rt1    rt5    p50    p90
                    0      1      0.00   0.00   0.00   0.00

HTTP Requests
-----
08:25:46.377 UTC POST /                               403 Forbidden
```

## Webhooks

Add webhook

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).

● <https://engraved-provoking-slimne...> (push)

Edit

Delete

This hook has never been triggered.

### Events:

Push events

5/12

## 9. Part E: Execution Flow (Stage-wise Explanation)

### Stage 1: Code Push

- Developer updates code in GitHub

### Stage 2: Webhook Trigger

- GitHub sends event to Jenkins

### Stage 3: Jenkins Pipeline Execution

#### Stage: Clone

- Pulls latest code from GitHub

#### Stage: Build

- Docker builds image using Dockerfile

#### Stage: Auth

- Jenkins logs into Docker Hub using stored token

#### Stage: Push

- Image pushed to Docker Hub

#### Stage 4: Artifact Ready

- Docker image available globally

### 10. Role of Same Host Agent

- Jenkins runs inside Docker
- Docker socket mounted:

`/var/run/docker.sock`

#### Effect

- Jenkins directly controls host Docker
- Builds and pushes images without separate agent

6/12

### 11. Observations

- Jenkins GUI simplifies CI/CD management
- GitHub acts as source + pipeline definition
- Docker ensures consistent builds
- Webhook enables automation

### 12. Result

Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline are maintained in GitHub
- Jenkins automatically detects changes
- Docker image is built on host agent
- Image is securely pushed to Docker Hub

## Experiment 9 : Ansible

### Theory

**Problem Statement:** Managing infrastructure manually across multiple servers leads to configuration drift, inconsistent environments, and time-consuming repetitive tasks. Scaling from one server to hundreds becomes nearly impossible with manual SSH-based administration.

### What is Ansible?

- Ansible is an open-source **automation tool** for **configuration management**, **application deployment**, and **orchestration**.
- It follows an **agentless** architecture, using SSH for Linux and WinRM for Windows.
- Uses YAML-based **playbooks** to define automation tasks.

*Ansible is software that enables cross-platform automation and orchestration at scale and has become the standard choice among enterprise automation solutions.*

## How Ansible Solves the Problem:

- **Agentless Architecture:** No software installation required on managed nodes
- **Idempotency:** Running playbooks multiple times yields same result
- **Declarative Syntax:** Describe desired state, not the steps to achieve it
- **Push-based:** Initiates changes from control node immediately

## Key Concepts

| Component            | Description                                                        |
|----------------------|--------------------------------------------------------------------|
| <b>Control Node</b>  | Machine with Ansible installed                                     |
| <b>Managed Nodes</b> | Target servers (no Ansible agent needed)                           |
| <b>Inventory</b>     | Defines the list of managed nodes (EC2 instances, servers, etc)    |
| <b>Playbooks</b>     | YAML files containing a sequence of automation steps.              |
| <b>Tasks</b>         | Individual actions in playbooks (e.g., installing a package).      |
| <b>Modules</b>       | Built-in functionality to perform tasks (e.g., yum, apt, service). |
| <b>Roles</b>         | Pre-defined reusable automation scripts.                           |

## How does Ansible work?

Ansible uses the concepts of control and managed nodes. It connects from the **control node**, any machine with Ansible installed, to the **managed nodes** sending commands and instructions to them. The units of code that Ansible executes on the managed nodes are called **modules**. Each module is invoked by a **task**, and an ordered list of tasks together forms a **playbook**. Users write playbooks with tasks and modules to define the desired state of the system. The managed machines are represented in a simplistic **inventory** file that groups all the nodes into different categories. Ansible leverages a very simple language, **YAML**, to define playbooks in a human-readable data format that is really easy to understand from day one.

*Even more, Ansible doesn't require the installation of any extra agents on the managed nodes so it's simple to start using it.*

Typically, the only thing a user needs is a terminal to execute Ansible commands and a text editor to define the configuration files.

## Benefits of using Ansible

- A free and open-source community project with a huge audience.
- Battle-tested over many years as the preferred tool of IT wizards.
- Easy to start and use from day one, without the need for any special coding skills.
- Simple deployment workflow without any extra agents.
- Includes some sophisticated features around modularity and reusability that come in handy as users become more proficient.
- Extensive and comprehensive official documentation that is complemented by a plethora of online material produced by its community.

*To sum up, Ansible is simple yet powerful, agentless, community-powered, predictable, and secure.*

## Part A

## Ansible Installation Instructions

### 1. Install via pip (Python Package Manager)

```
# Install Ansible globally
pip install ansible
```

```
# Verify installation
ansible --version
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/aditya/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/aditya/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

## Ansible Demo

*Ansible demo with docker container as servers*

3/8

## Create Docker image and test ssh login

*First create ssh key-pair and then create custom ubuntu-server image with open-ssh configured.*

### Testing SSH Key Pair Login with Docker and WSL

Here's a step-by-step guide to create and test SSH key pair authentication between WSL and a Docker container running Ubuntu with OpenSSH server:

### 1. Create SSH Key Pair in WSL

First, generate an SSH key pair in your WSL environment:

```
# Generate RSA key pair (accept defaults when prompted)
ssh-keygen -t rsa -b 4096
```

```
aditya@Aditya:/mnt/c/Users/Arundun$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/aditya/.ssh/id_rsa):
Created directory '/home/aditya/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/aditya/.ssh/id_rsa
Your public key has been saved in /home/aditya/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:diAMsBD/I8yjpz0ylledo00L960nw/p9yRgcWZcbwIA aditya@Aditya
The key's randomart image is:
+---[RSA 4096]-----+
|o.... ..o.. .|
| o . E  o + |
| o  o + . o |
| o . . + . . |
| = + + S . |
| . = o * . |
| . + + + . |
| +=. + +oo + |
|==oo+=o. |
+---[SHA256]-----+
aditya@Aditya:/mnt/c/Users/Arundun$
```

```
# This creates:
# Private key: ~/.ssh/id_rsa
# Public key: ~/.ssh/id_rsa.pub
# copy keys to current directory to be added to docker images
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

*Here's a simple breakdown of where each key should be placed:*

| File                    | Location (Machine)                                | Purpose                                                  |
|-------------------------|---------------------------------------------------|----------------------------------------------------------|
| id_rsa (Private Key)    | Your local machine (where you run ssh or Ansible) | Used to authenticate when connecting to server           |
| id_rsa.pub (Public Key) | Remote server (inside ~/.ssh/authorizedkeys)      | Grants access to anyone who has the matching private key |

## 2. Create Dockerfile for Ubuntu SSH Server

Create a Dockerfile with the following content:

```
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys (note: this is not secure for production!)
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

# Set proper permissions for keys
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# Fix for SSH login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

# Expose SSH port
EXPOSE 22

# Start SSH service when container starts
CMD ["/usr/sbin/sshd", "-D"]
```

## 3. Build the Docker Image

```
# Copy your public key to the current directory
cp ~/.ssh/id_rsa.pub .
```

```
# Build the Docker image
docker build -t ubuntu-server .
```

```
aditya@aditya: /mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ docker build -t ubuntu-server .
[+] Building 33.1s (6/14) docker:default
=> [internal] load build definition from dockerfile 0.0s
=> => transferring dockerfile: 1.13kB 0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 1.2s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194eb 7.7s
=> => resolve docker.io/library/ubuntu:latest@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc 0.0s
=> => sha256:b40150c1c2717d324cdb17278c8efdfa4dfcd2ffe083e976f0bcedf31115f081 29.73MB / 29.73MB 6.8s
=> => extracting sha256:b40150c1c2717d324cdb17278c8efdfa4dfcd2ffe083e976f0bcedf31115f081 0.9s
=> [internal] load build context 0.0s
=> => transferring context: 4.20kB 0.0s
=> [ 2/10] RUN apt update -y 14.3s
=> [ 3/10] RUN apt install -y python3 python3-pip openssh-server 9.8s
=> => # Get:39 http://archive.ubuntu.com/ubuntu noble/main amd64 ucf all 3.0043+nmu1 [56.5 kB]
=> => # Get:40 http://archive.ubuntu.com/ubuntu noble/main amd64 libwrap0 amd64 7.6.q-33 [47.9 kB]
=> => # Get:41 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-server amd64 1:9.6p1-3ubuntu13.16 [51
=> => # 0 kB]
=> => # Get:42 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 perl-modules-5.38 all 5.38.2-3.2ubuntu0.2 [31
=> => # 10 kB]
```

```
# Remove the public key from build directory (optional)
rm id_rsa.pub
```

## 4. Run the Docker Container

```
# Run the container with port mapping
docker run -d -p --rm 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
66b18cc1fdd8aed739b46d7fbb4cc2ab4dc03a9ea4ceabd3d1512fa82f6b4b2e
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ |
```

## 5. Find the Container IP Address

```
# Get the container's IP address
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server
172.17.0.2
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ |
```

Note this IP address (e.g., 172.17.0.2)

## 6. Test SSH Connections

### Test password authentication:

```
ssh root@localhost -p 2222
# OR
ssh root@172.17.0.2
# If ssh keys are working it likely not use them, Else, When prompted, enter password: "password"
```

### Test key-based authentication:

```
ssh -i ~/.ssh/id_rsa root@localhost -p 2222
# Should log in without password prompt
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ ssh root@172.17.0.2
ssh: connect to host 172.17.0.2 port 22: No route to host
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ ssh -i ~/.ssh/id_rsa root@localhost -p 2222
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:1L7t0EGMSIXKL+tTNjEJ+a8KlJlNvb0jqzHCt+48xI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@66b18cc1fdd8:~# |
```

## 7. Alternative: Using Container IP Directly

If you prefer to use the container's IP instead of port mapping:

```
# Using the IP you got from docker inspect
ssh root@172.17.0.2
# or with key:
ssh -i ~/.ssh/id_rsa root@172.17.0.2
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ ssh -i ~/.ssh/id_rsa root@localhost -p 2222
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Fri May 1 12:26:28 2026 from 172.17.0.1
root@66b18cc1fdd8:~# |
```

## Verification Steps

1. Confirm password authentication works
2. Confirm key-based authentication works without password prompt



3. Verify you can access port 8221 (though nothing is running there yet)
4. Check logs if any issues occur: `docker logs ssh-test`

## Clean Up

When done testing:

```
docker stop ssh-test
docker rm ssh-test
```

This setup demonstrates both SSH key-based authentication and password authentication working together in a Docker container.

4/8

## Ansible with Docker Exercise

Using docker image `ubuntu-server` created in previous part run 4 test servers named `server1` to `serve4`

### Step 1: Start multiple containers to act as server (to be configured by ansible)

```
for i in {1..4}; do
  echo -e "\n Creating server${i}\n"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo -e "IP of server${i} is $(docker inspect -f
'{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i}) "
done
```

#### Step X: To stop all servers (if required)

```
for i in {1..4}; do
  docker stop server${i}
done
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ for i in {1..4}; do
  docker run -d -p 220${i}:22 --name server${i} ubuntu-server
done
359642d94b187ece6372cc5a8762e81d2bb06bde6dbb2a96c34f8fff52c83550
375d63b73817c6b69efde0361384ab27b54de2f674372f0daf2f96d763930b41
ddb34e87ff938e3afdbf434da0d60ed1bfff7320d9d8df558636cc4f7c73852d3
6d4cb7aa3c7ee29d080258377960e9fa5295c43ef45fbd2e98e4ddb06707cfa
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ |
```

### Step 2: Create Ansible Inventory

below script will create an update inventory.ini with updated docker container IPs, review this file and you can create your own if needed.

```
# Get container IPs
echo "[servers]" > inventory.ini
for i in {1..4}; do
  docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

# Add inventory variables
cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ ansible all -i inventory.ini -m ping
The authenticity of host '[localhost]:2203 ([127.0.0.1]:2203)' can't be established.
ED25519 key fingerprint is SHA256:1l7t0EGMSIXKl+tTNjEJ+aBKLjlnvb0qjzHCt+48xi.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
  ~/.ssh/known_hosts:4: [hashed name]
  ~/.ssh/known_hosts:5: [hashed name]
The authenticity of host '[localhost]:2204 ([127.0.0.1]:2204)' can't be established.
ED25519 key fingerprint is SHA256:1l7t0EGMSIXKl+tTNjEJ+aBKLjlnvb0qjzHCt+48xi.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
  ~/.ssh/known_hosts:4: [hashed name]
  ~/.ssh/known_hosts:5: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? server2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
yes
server3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
}
```

## Step 3: review content of inventory.ini

```
cat inventory.ini
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ cat inventory.ini
[servers]
server1 ansible_host=localhost ansible_port=2201
server2 ansible_host=localhost ansible_port=2202
server3 ansible_host=localhost ansible_port=2203
server4 ansible_host=localhost ansible_port=2204

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$
```

## Step 4: Test Connectivity

```
# Manual SSH test
ssh -i $(pwd)/id_rsa root@172.17.0.3
## above on will work on linux and may cause problems on NTFS file systems, then try
ssh -i ~/.ssh/id_rsa root@172.17.0.3

# Ansible ping test
ansible all -i inventory.ini -m ping
OR
ansible all -i inventory.ini -m ping -vvv
```

## Step 5: Create Playbook (update.yml)

The yaml file should start with three dash only ---

```
--- # it should start with three dash only
- name: Update and configure servers
  hosts: all
  become: yes
```

```
tasks:
  - name: Update apt packages
    apt:
      update_cache: yes
      upgrade: dist

  - name: Install required packages
    apt:
      name: ["vim", "htop", "wget"]
      state: present

  - name: Create test file
    copy:
      dest: /root/ansible_test.txt
      content: "Configured by Ansible on {{ inventory_hostname }}"
```

## Step 6: Run Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

## Step 7: Verify Changes

```
# Using Ansible
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"

# Manually via Docker
for i in {1..4}; do
```

```
docker exec server${i} cat /root/ansible_test.txt
done
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ ansible-playbook -i inventory.ini update.yml
PLAY [Update and configure servers] *****
TASK [Gathering Facts] *****
The authenticity of host '[localhost]:2204 ([127.0.0.1]:2204)' can't be established.
ED25519 key fingerprint is SHA256:1l7w0EG0S1XkL+tHjE3+haBkLj1Uvb0gJzHCt+48xi.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
  ~/.ssh/known_hosts:4: [hashed name]
  ~/.ssh/known_hosts:5: [hashed name]
  ~/.ssh/known_hosts:6: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? ok: [server3]
ok: [server2]
ok: [server1]
```

## Step 8: Cleanup

```
# Stop and remove containers
for i in {1..4}; do docker rm -f server${i}; done
```

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ for i in {1..4}; do docker rm -f server${i}; done
server1
server2
server3
server4
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab9$ |
```

The workflow is now:

1. Setup SSH keys → 2. Build image → 3. Launch containers → 4. Create inventory → 5. Test → 6. Run playbook → 7. Verify → 8. Cleanup

5/8

## The Need for Ansible in Server Management

Ansible is a powerful automation tool that addresses several critical challenges in server management:

1. **Scalability**: Managing multiple servers manually becomes impractical as infrastructure grows.
2. **Consistency**: Ensures identical configurations across all servers, reducing "works on my machine" issues.
3. **Efficiency**: Automates repetitive tasks, saving time and reducing human error.
4. **Idempotency**: Operations can be run multiple times without causing unintended changes.
5. **Infrastructure as Code**: Configuration is version-controlled and documented.

### Key Features of Ansible

1. **Agentless** – Uses SSH (no need to install software on managed nodes).
2. **Idempotent** – Ensures desired state without repeating changes unnecessarily.
3. **YAML-Based Playbooks** – Simple, human-readable automation scripts.
4. **Modules** – 3000+ built-in modules for cloud, containers, networking, etc.
5. **Push-Based** – Executes tasks from a control machine.
6. **Infrastructure as Code (IaC)** – Supports declarative configuration management.

### Why Choose Ansible?

- **No Agents**: Uses SSH/WinRM.
- **Extensible**: Custom modules via Python.
- **Multi-Platform**: Linux, Windows, cloud (AWS/Azure), networking devices.

For troubleshooting, check:

```
ansible-doc -l          # List all modules
ansible-doc apt         # View module docs
# do try
ansible-doc -l | grep aws
```

**You can try this using ubuntu running in VM instead of docker container or your another system.**

Try to use *Ansible* to configure an *EC2 instance* on *AWS*.

7/8

## Optional Part B

### 1. Set up a local environment:

- **Install Ansible:** If you're on Linux or macOS, you can install Ansible with:

```
sudo apt-get install ansible # For Ubuntu
brew install ansible        # For macOS
```

For Windows, you can install Ansible using Windows Subsystem for Linux (WSL) or use the Windows version of Ansible through Cygwin.

- **Create a Virtual Machine or Use Containers:** You can use tools like VirtualBox, Vagrant, or Docker to spin up virtual machines or containers to test Ansible playbooks locally.
- For Vagrant:

```
vagrant init ubuntu/bionic64
vagrant up
```

- **Set Up Local Inventory:** Create an inventory file (`inventory.ini`) listing the hosts you want to target. For testing locally, you can use `localhost`:

```
[localhost]
localhost ansible_connection=local
```

### 2. Write a Simple Playbook:

A playbook is where you'll define the tasks you want Ansible to perform. Here's an example of a simple playbook that installs `nginx` on your machine:

Create a file `install_nginx.yml`

*The yaml file should start with three dash only ---*

```
--- # it should start with three dash only
- name: Install Nginx on localhost
  hosts: local
  become: yes
  tasks:
  - name: Install nginx package
    apt:
      name: nginx
      state: present
```

### 3. Run the Playbook:

Test the playbook by running it on the local host. Use the command:

```
ansible-playbook -i inventory.ini install_nginx.yml
```

### 4. Test Other Tasks:

- Test various modules (like `copy`, `file`, `template`, `command`, etc.) to get hands-on experience.
- Create playbooks for different tasks (e.g., managing services, creating users, configuring files).

## Experiment 10: SonarQube — Static Code Analysis

### Theory

**Problem Statement:** Code bugs and security issues are often found too late — during testing or even after deployment. Manual code reviews are slow, inconsistent, and don't scale as teams grow.

**What is SonarQube?** SonarQube is an open-source platform that automatically scans your source code for bugs, security vulnerabilities, and maintainability issues — without running the code. This is called **static analysis**.

**How SonarQube Solves the Problem:**

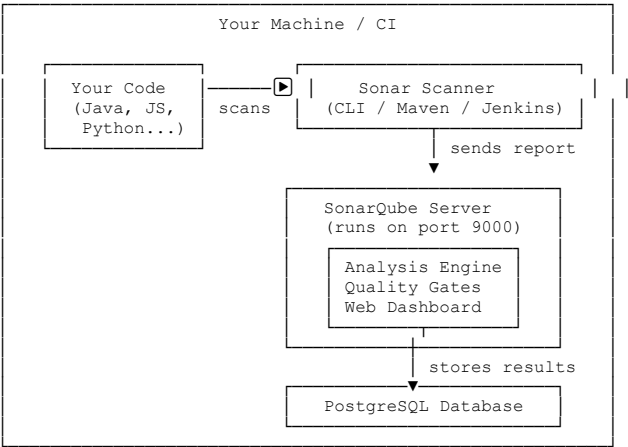
- Scans code on every commit, giving immediate feedback
- Enforces **Quality Gates** — pass/fail checks before code is deployed
- Tracks **Technical Debt** — how long it would take to fix all issues
- Supports 20+ programming languages
- Provides a visual dashboard for trends over time

**Key Terms:**

| Term           | What it means                                             |
|----------------|-----------------------------------------------------------|
| Quality Gate   | A set of rules; code must pass before deployment          |
| Bug            | Code that will likely break or behave incorrectly         |
| Vulnerability  | A security weakness in the code                           |
| Code Smell     | Code that works but is poorly written or hard to maintain |
| Technical Debt | Estimated time to fix all issues                          |
| Coverage       | Percentage of code tested by unit tests                   |
| Duplication    | Repeated code blocks (copy-paste)                         |

**Lab Architecture**

SonarQube has *two separate components* — a *Server* (the brain) and a *Scanner* (the worker). Both are required.



Your Machine / CIYour Code(Java, JS, Python...)Sonar Scanner(CLI / Jenkins)scansreportSonarQube Server (9000)Analysis EngineQuality GatesWeb DashboardPostgreSQL DBstoresUserview

## Part 1: SonarQube Server — "The Brain"

### What it is

The **SonarQube Server** is a web application that:

- Stores all analysis results in a database
- Applies code quality rules (bugs, vulnerabilities, smells)
- Shows a web dashboard at `http://localhost:9000`
- Enforces Quality Gates (pass/fail decisions)
- Tracks history and trends over time

### Think of it like:

*A **teacher / examiner** — it receives work, grades it, and shows the report.*

### What runs inside the server:

- Analysis Engine (applies the rules)
- Web UI (dashboard you browse to)
- PostgreSQL database (stores everything)

3/21

## Part 2: Sonar Scanner — "The Worker"

### What it is

The **Sonar Scanner** is a command-line tool that:

- Reads your source code files
- Detects issues based on the server's rule set
- Sends an analysis report to the SonarQube Server

### Think of it like:

*A **student writing an exam** — it does the work and submits it.*

### Scanner types (you don't always use the CLI):

| Scanner Type                                  | When to use               |
|-----------------------------------------------|---------------------------|
| sonar-scanner (CLI)                           | Any project, run manually |
| Maven plugin ( <code>mvn sonar:sonar</code> ) | Java/Maven projects       |
| Gradle plugin                                 | Java/Gradle projects      |
| Jenkins integration                           | CI/CD pipelines           |
| GitHub Actions                                | Automated PRs             |

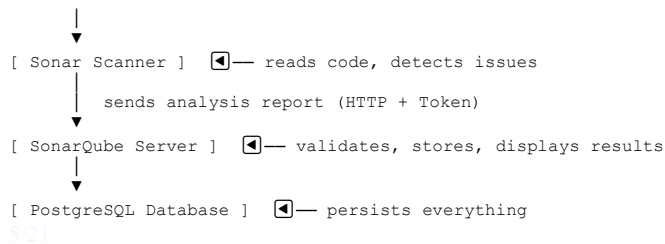
4/21

## Why Both Are Required

| Only Server installed                       | Only Scanner installed                        |
|---------------------------------------------|-----------------------------------------------|
| Dashboard is empty<br>No code gets analyzed | Nowhere to send results<br>Analysis is wasted |
| Both installed together                     |                                               |
| Full pipeline works ✓                       |                                               |

### Flow summary:

[ Your Code ]



## Quick Comparison Table

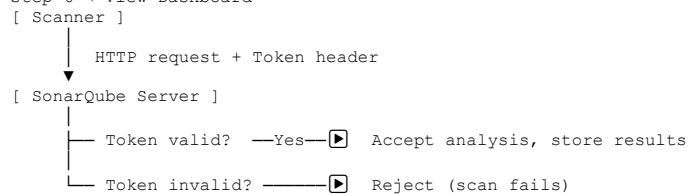
| Feature  | SonarQube Server          | Sonar Scanner    |
|----------|---------------------------|------------------|
| Type     | Server application        | CLI / plugin     |
| Role     | Store & display results   | Analyze code     |
| Web UI   | Yes (port 9000)           | No               |
| Runs on  | Server / Docker container | Dev machine / CI |
| Required | Yes                       | Yes              |

6/21

## How the Token Works

The scanner does **not** use a username/password. It uses a **token** for authentication.

Step 1 → Start SonarQube Server  
 Step 2 → Open Web UI at <http://localhost:9000>  
 Step 3 → Generate Token in UI — you copy this  
 Step 4 → Run Scanner, passing the token  
 Step 5 → Server validates token, stores results  
 Step 6 → View Dashboard



### Token is:

- Generated once in the **Server UI**
- Used by the **Scanner** on every scan
- Shown only once — copy it immediately

7/21

## Hands-on Lab

8/21

### Step 1: Start the SonarQube Server

We will use Docker Compose to start both the SonarQube server and its PostgreSQL database together.

Create a file called `docker-compose.yml`:

```
# docker-compose.yml
# This file starts two containers:
# 1. sonar-db    → PostgreSQL database (stores SonarQube data)
# 2. sonarqube   → The SonarQube server (web UI + analysis engine)

version: '3.8'

services:

  # --- Database ---
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar      # DB username
      POSTGRES_PASSWORD: sonar  # DB password
      POSTGRES_DB: sonarqube    # DB name
      POSTGRES_HOST_AUTH_METHOD: trust # allow connections without password (for simplicity in this lab)
    volumes:
      - sonar-db-data:/var/lib/postgresql/data # persist data across restarts
    networks:
      - sonarqube-lab

  # --- SonarQube Server ---
  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000" # access dashboard at http://localhost:9000
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db # wait for DB to start first
    networks:
      - sonarqube-lab

# Named volumes (Docker manages storage location)
volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

# Isolated network so containers can talk to each other by name
networks:
  sonarqube-lab:
    driver: bridge
```

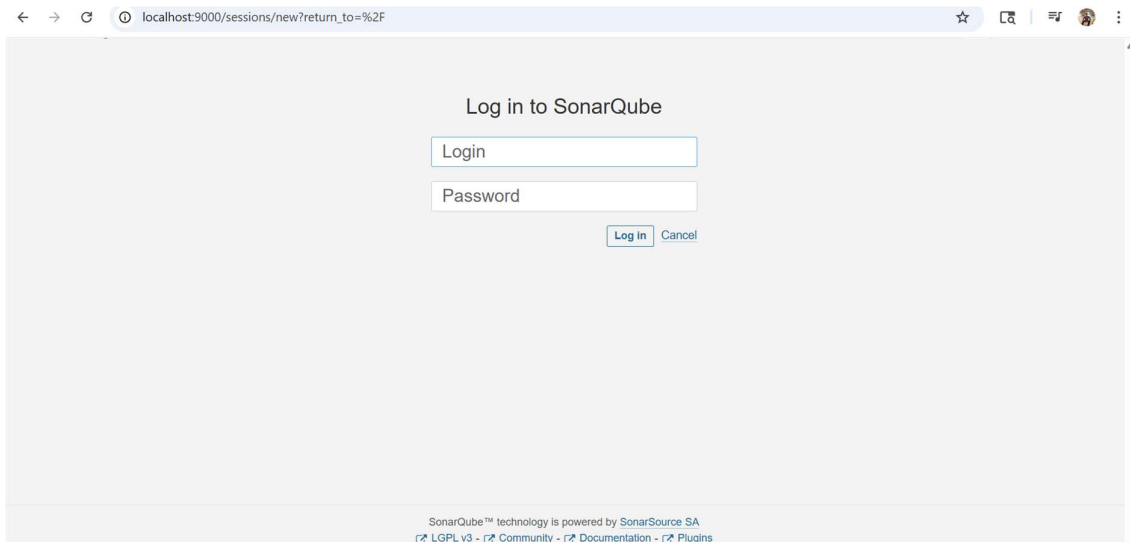
Start both containers:

```
docker-compose up -d
```

```
# Watch the logs until you see "SonarQube is up"
docker-compose logs -f sonarqube
```

Once started, open `http://localhost:9000` in your browser. Default login: **admin / admin** (you will be asked to change this on first login).





```
aditya@Aditya: /mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab10$ docker-compose up -d
WARN[0000] /mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab10/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] up 28/28
[*] up 34/34
postgres:13 Pulled 96.2s
Image postgres:13 Pulled 96.2s
Image sonarqube:lts-community Pulled 132.5s
Network lab10_sonarqube-lab Created 0.1s
Volume lab10_sonar-data Created 0.0s
Volume lab10_sonar-extensions Created 0.0s
Volume lab10_sonar-db-data Created 0.0s
Container sonar-db Created 0.6s
Container sonarqube Created 0.2s
```

9/21

## Step 2: Create a Sample Java App with Code Issues

We will create a simple Java class that intentionally contains bugs, vulnerabilities, and code smells — so SonarQube has something to detect.

```
# Create the project folder structure
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

Create `src/main/java/com/example/Calculator.java`:

```
package com.example;

public class Calculator {

    // — BUG: Division by zero is not handled —————
    // If someone calls divide(5, 0), this will crash at runtime.
    public int divide(int a, int b) {
        return a / b;
    }

    // — CODE SMELL: Unused variable —————
    // The variable 'unused' is declared but never used.
    // SonarQube flags this as unnecessary clutter.
    public int add(int a, int b) {
        int result = a + b;
        int unused = 100; // ← code smell: delete this line
        return result;
    }

    // — VULNERABILITY: SQL Injection risk —————
    // Building a query by concatenating user input is dangerous.
    // An attacker could pass: "1 OR 1=1" and get all users.
    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }
}
```

```

// — CODE SMELL: Duplicated code —————
// The two methods below do exactly the same thing.
// This is copy-paste code and should be a single method.
public int multiply(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;
    }
    return result;
}

public int multiplyAlt(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a; // ← exact duplicate of multiply()
    }
    return result;
}

// — BUG: Null pointer risk —————
// If 'name' is null, calling .toUpperCase() will throw
// a NullPointerException at runtime.
public String getName(String name) {
    return name.toUpperCase();
}

// — CODE SMELL: Empty catch block —————
// The exception is caught but silently ignored.
// This hides errors and makes debugging very hard.
public void riskyOperation() {
    try {
        int x = 10 / 0;
    } catch (Exception e) {
        // ← never leave catch blocks empty
    }
}
}

```

## Create pom.xml (Maven build file):

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <!-- Project identity -->
    <groupId>com.example</groupId>
    <artifactId>sample-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>

        <!-- SonarQube connection settings -->
        <sonar.projectKey>sample-java-app</sonar.projectKey>
        <sonar.host.url>http://localhost:9000</sonar.host.url>
        <!-- Replace with your actual token (generated in Step 3) -->
        <sonar.login>YOUR_TOKEN_HERE</sonar.login>
    </properties>

    <dependencies>
        <!-- JUnit for unit tests -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- This plugin lets us run: mvn sonar:sonar -->
            <plugin>
                <groupId>org.sonarsource.scanner.maven</groupId>
                <artifactId>sonar-maven-plugin</artifactId>
                <version>3.9.1.2184</version>
            </plugin>
        </plugins>
    </build>

</project>

```

10/21

### Step 3: Generate a Token (Manual UI Step)

The Scanner needs a token to authenticate with the server. You generate this in the web UI.

1. Open `http://localhost:9000`
2. Log in as admin
3. Click your user icon (top right) → "My Account"
4. Click the "Security" tab
5. Under "Generate Tokens", type a name: scanner-token
6. Click "Generate"
7. Copy the token immediately – it is shown only once!  
It looks like: `sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX`

**Important:** Store the token somewhere safe. You cannot retrieve it again after closing the page.

11/21

### Step 4: Run the Scanner

```
aditya@Aditya:/mnt/c/users/Arundun/Desktop/Containers/Container/LAB/lab10$ mvn sonar:sonar
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom (3.9 kB at 7.6 kB/s)
)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom (13 kB at 189 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom (26 kB at 366 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/10/apache-10.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/10/apache-10.pom (15 kB at 190 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.jar (25 kB at 328 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.pom (6.4 kB at 85 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/23/maven-plugins-23.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/23/maven-plugins-23.pom (9.2 kB at 119 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/22/maven-parent-22.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/22/maven-parent-22.pom (30 kB at 407 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/11/apache-11.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/11/apache-11.pom (15 kB at 239 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/2.4/maven-install-plugin-2.4.jar (27 kB at 421 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/2.7/maven-deploy-plugin-2.7.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/2.7/maven-deploy-plugin-2.7.pom (5.6 kB at 66 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/2.7/maven-deploy-plugin-2.7.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/2.7/maven-deploy-plugin-2.7.jar (27 kB at 323 kB/s)
)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-site-plugin/3.3/maven-site-plugin-3.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-site-plugin/3.3/maven-site-plugin-3.3.pom (21 kB at 320 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/24/maven-plugins-24.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/24/maven-plugins-24.pom (11 kB at 174 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/23/maven-parent-23.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/23/maven-parent-23.pom (33 kB at 466 kB/s)
```

`sonar.sourceEncoding=UTF-8`

### Run scanner using docker

Then run the scanner using Docker (no local install needed):

```
# Run the scanner container, mounting your project folder inside it
docker run --rm \
  --network sonarqube-lab \
  -e SONAR_TOKEN="YOUR_TOKEN" \
  -v "$(pwd):/usr/src" \
  sonarsource/sonar-scanner-cli \
  -Dsonar.host.url=http://sonarqube:9000 \
  -Dsonar.projectBaseDir=/usr/src \
  -Dsonar.projectKey=sample-java-app
```

*you might need to change network name based on actual network name used by sonar-server*

```
docker network ls
# or
docker inspect sonarqube | grep -i network
```

#### Flag

`--rm`

`--network`

`-e SONAR_TOKEN`

`-v`

#### Purpose

auto delete container

connect to SonarQube

authentication

mount source code

| Flag                                | Purpose                               |
|-------------------------------------|---------------------------------------|
| <code>-Dsonar.host.url</code>       | server URL                            |
| <code>-Dsonar.projectBaseDir</code> | project root                          |
| <code>-Dsonar.projectKey</code>     | project identifier to store on server |

**Note:** We use `http://sonarqube:9000` (container name, not localhost) because the scanner container is on the same Docker network as the server container.

12/21

## Step 5: View Results in the Dashboard

After the scan finishes, open the dashboard:

`http://localhost:9000/dashboard?id=sample-java-app`

You should see something like:

| sample-java-app - Dashboard                                                                     |                |                  |
|-------------------------------------------------------------------------------------------------|----------------|------------------|
| Bugs<br>5                                                                                       | Vulnerab.<br>1 | Code Smells<br>8 |
| Coverage: 0%      Duplications: 2 blocks<br>Technical Debt: ~1h 30min<br>Quality Gate: X FAILED |                |                  |

Click on any number to see the exact line and reason for each issue.

You can also query results via the API:

# List all bugs found in the project (returns JSON)

```
curl -u admin:YOUR_TOKEN \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=BUG"
```

13/21

## Step 6: Integrate with Jenkins (CI/CD)

Once SonarQube is working locally, you can add it to a Jenkins pipeline so every code commit is automatically scanned.

```
// Jenkinsfile
// This pipeline checks out code, scans it with SonarQube,
// waits for the Quality Gate result, then builds and deploys.
```

```

Container > LAB > lab10 > jenkinsfile
1 pipeline {
2   agent any
3
4   environment {
5     SONAR_HOST_URL = 'http://sonarqube:9000'
6     // 'sonar-token' is a Jenkins credential - store your token there
7     SONAR_TOKEN = credentials('sonar-token')
8   }
9
10  stages {
11
12    stage('Checkout') {
13      steps {
14        // Pull the latest code from the repository
15        checkout scm
16      }
17    }
18
19    stage('SonarQube Analysis') {
20      steps {
21        // This injects SonarQube server URL + authentication token
22        withSonarQubeEnv('SonarQube') {
23
24          // mvn = Maven tool (used mainly for Java projects)
25          // clean = remove old build files
26          // verify = compile code + run tests
27          // sonar:sonar = send analysis report to SonarQube server
28          sh 'mvn clean verify sonar:sonar'
29        }
30      }
31    }
32
33    stage('Quality Gate') {
34      steps {
35        // Wait for SonarQube to finish processing the report.
36        // If the Quality Gate fails, the pipeline stops here.
37        timeout(time: 5, unit: 'MINUTES') {
38          waitForQualityGate abortPipeline: true
39        }
40      }
41    }
42
43    stage('Build') {
44      steps {
45        // Only runs if Quality Gate passed
46        sh 'mvn package'
47      }
48    }
49
50    stage('Deploy') {
51      steps {
52        sh 'docker build -t sample-app .'
53        sh 'docker run -d -p 8080:8080 sample-app'
54      }
55    }
56  }
57 }
58

```

## Pipeline flow:

Checkout → Scan → Quality Gate check → Build → Deploy

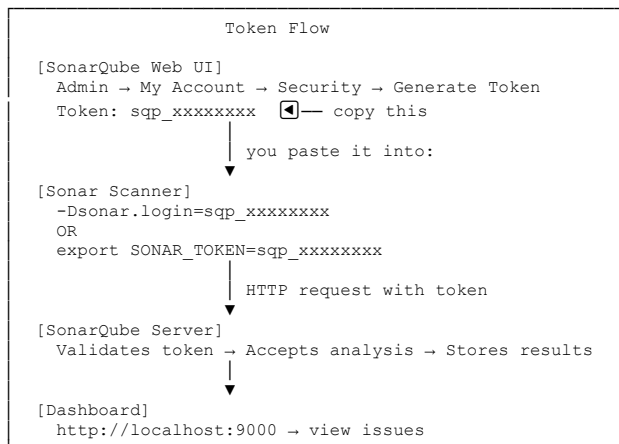
|

FAIL? Pipeline stops.

Code does not get deployed.

14/21

## Token Generation — Step-by-Step Summary



Three ways to pass the token to the scanner:

```
# Option A - directly in the command
sonar-scanner -Dsonar.login=YOUR_TOKEN

# Option B - environment variable (cleaner, preferred in CI)
export SONAR_TOKEN=YOUR_TOKEN
sonar-scanner -Dsonar.host.url=http://localhost:9000

# Option C - Maven flag
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

Comparative Analysis & Summary

Tool Comparison Matrix

| Feature          | Jenkins          | Ansible                   | Chef              | SonarQube       |
|------------------|------------------|---------------------------|-------------------|-----------------|
| Primary Purpose  | CI/CD Automation | Config Management         | Config Management | Code Quality    |
| Architecture     | Master-Agent     | Agentless                 | Client-Server     | Client-Server   |
| Language         | Java / Groovy    | YAML                      | Ruby              | Java            |
| Learning Curve   | Moderate         | Low                       | High              | Low             |
| Setup Complexity | Moderate         | Simple                    | Complex           | Simple          |
| Use Case         | Build, Deploy    | Test, Infrastructure Code | as Enterprise CM  | Static Analysis |

Key Differences

**Jenkins vs. Configuration Management:** Use Jenkins to orchestrate pipelines (build → test → deploy). Use Ansible or Chef to maintain server configuration. In practice, Jenkins calls Ansible/Chef as part of the deployment step.

Ansible vs. Chef:

| Aspect         | Ansible            | Chef              |
|----------------|--------------------|-------------------|
| Complexity     | Simple, YAML       | Complex, Ruby     |
| Agent required | No (agentless)     | Yes (chef-client) |
| Best for       | Small–medium teams | Large enterprise  |

**SonarQube in CI/CD:** Scan on every pull request and every nightly build. Use Quality Gates to block deployments when code quality drops below your threshold.

Commands Quick Reference

```
SonarQube
# Start server
docker-compose up -d

# Run scan with Maven
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN

# Run scan with CLI
sonar-scanner -Dsonar.host.url=http://localhost:9000 -Dsonar.login=YOUR_TOKEN

# Query API for bugs
curl -u admin:YOUR_TOKEN \
```

<http://localhost:9000/api/issues/search?projectKeys=myapp&types=BUG>

```
cd /tmp/9000/; mvn sonar:sonar -Dsonar.login=sqa_8986cca75d15f9f5fb15dee3266186d8b326c67
[INFO] Scanning for projects...
[WARNING] The artifact org.codehaus.mojo:sonar-maven-plugin:jar:4.0.0.4121 has been relocated to org.sonarsource.scanner.maven:sonar-maven-plugin:jar:4.0.0.4121: SonarQube plugin was moved to SonarSource orga
nisation
[INFO]
[INFO] -----< org.apache.maven:standalone-pom -----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 0.471 s
[INFO] Finished at: 2026-05-01T16:44:31Z
```

## Jenkins

```
# Start Jenkins container
docker run -d -p 8080:8080 -v jenkins-data:/var/jenkins_home jenkins/jenkins:lts
```

```
# Example pipeline stage
pipeline {
    agent any
    stages {
        stage('Build') { steps { sh 'mvn package' } }
    }
}
```

## Ansible

```
ansible all -i inventory -m ping
ansible-playbook -i inventory playbook.yml
ansible webserver -m apt -a "name=nginx state=present"
```

## Chef

```
knife node list
knife cookbook upload webapp
chef-client --local-mode --runlist 'recipe[webapp]'
```

17/21

## Best Practices

### Security:

- Never hardcode tokens or passwords in source files — use environment variables or a secrets manager
- Use HTTPS for all SonarQube server communications in production
- Apply least-privilege: give the scanner token only the permissions it needs

### Code Quality:

- Set Quality Gates to block merges when coverage drops below 80%
- Scan on every pull request, not just nightly
- Fix issues as they appear — do not let technical debt accumulate

### Infrastructure as Code:

- Version all config files in Git
- Use modular Ansible roles and Chef cookbooks
- Test infrastructure code with Molecule (Ansible) or Test Kitchen (Chef)

### CI/CD Optimization:

- Cache Maven / npm dependencies between builds
- Run SonarQube analysis in parallel with unit tests where possible
- Configure the pipeline to fail fast on Quality Gate failures

18/21

## Summary:

*SonarQube = the analysis platform that stores and displays results.*

*Sonar Scanner = the tool that reads your code and sends the report to that platform.*

*Both are required. The Scanner needs a Token to talk to the Server. Quality Gates let you automatically block bad code from being deployed.*

19/21

# Experiment 12: Study and Analyse Container Orchestration using Kubernetes

## Objective

Learn why Kubernetes is used, its basic concepts, and how to deploy, scale, and fix apps using Kubernetes commands.

## Why Kubernetes over Docker Swarm?

| Reason               | Explanation                                 |
|----------------------|---------------------------------------------|
| Industry standard    | Most companies use Kubernetes               |
| Powerful scheduling  | Automatically decides where to run your app |
| Large ecosystem      | Many tools and plugins available            |
| Cloud-native support | Works on AWS, Google Cloud, Azure, etc.     |

2/14

## Core Kubernetes Concepts (Simple Explanation)

| Docker Concept  | Kubernetes Equivalent | What it means                                  |
|-----------------|-----------------------|------------------------------------------------|
| Container       | <b>Pod</b>            | A pod is a group of one or more containers. Sm |
| Compose service | <b>Deployment</b>     | Describes how your app should run (e.g., 2 cop |
| Load balancing  | <b>Service</b>        | Exposes your app to the outside world or other |
| Scaling         | <b>ReplicaSet</b>     | Ensures a certain number of pod copies are alw |

3/14

## Hands-On Lab (Using k3d or Minikube)

**Note:** We assume you already have `kubectl` and a cluster (*k3d* or *Minikube*) installed. If not, ask your instructor.

4/14

## Task 1: Create a Deployment

A **deployment** tells Kubernetes:

- Which container image to use (e.g., WordPress)
- How many copies (replicas) to run
- How to identify the pods (labels)

### Step 1: Create a file `wordpress-deployment.yaml`

```
# wordpress-deployment.yaml
apiVersion: apps/v1      # Which Kubernetes API to use
kind: Deployment         # Type of resource
metadata:
  name: wordpress        # Name of this deployment
spec:
  replicas: 2            # Run 2 identical pods
  selector:
    matchLabels:
      app: wordpress     # Pods with this label belong to this deployment
  template:              # Template for the pods
```



```

metadata:
  labels:
    app: wordpress # Label applied to each pod
spec:
  containers:
  - name: wordpress
    image: wordpress:latest # Docker image
    ports:
    - containerPort: 80 # Port inside the container

```

## Step 2: Apply the deployment

```
kubectl apply -f wordpress-deployment.yaml
```

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ |

```

**What happens?** Kubernetes creates 2 pods running WordPress.

5/14

## Task 2: Expose the Deployment as a Service

Pods are **temporary** (they can be deleted or recreated). A **Service** gives them a fixed IP and exposes them to the outside.

### Step 1: Create a file `wordpress-service.yaml`

```

# wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort # Exposes service on a port of each node (VM)
  selector:
    app: wordpress # Send traffic to pods with this label
  ports:
  - port: 80 # Service port
    targetPort: 80 # Pod port
    nodePort: 30007 # External port (range: 30000-32767)

```

### Step 2: Apply the service

```
kubectl apply -f wordpress-service.yaml
```

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl apply -f wordpress-service.yaml
service/wordpress-service created
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$

```

## Task 3: Verify Everything

Check if pods are running:

```
kubectl get pods
```

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-848c87c44f-2vplq         1/1     Running   0           2m26s
wordpress-848c87c44f-64rqk         1/1     Running   0           2m26s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$

```

Check the service:

```
kubectl get svc
```

```

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1     <none>       443/TCP          5m51s
wordpress-service   NodePort    10.105.135.228 <none>       80:30007/TCP     48s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$

```

## Access WordPress in your browser

```
http://<node-ip>:30007
```

## How to find node-ip in k3d/minikube?

- **Minikube:** `minikube ip`
- **k3d:** Usually `localhost`

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ minikube start
minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
🌟 Automatically selected the docker driver. Other choices: ssh, none
🚧 Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🚧 Using Docker driver with root privileges
🚧 For an improved experience it's recommended to use Docker Engine instead of Docker Desktop.
Docker Engine installation instructions: https://docs.docker.com/engine/install/#server
🔥 Starting "minikube" primary control-plane node in "minikube" cluster
📡 Pulling base image v0.0.50 ...
📦 Downloading Kubernetes v1.35.1 preload ...
> gcr.io/k8s-minikube/kicbase...: 49.75 KiB / 519.58 MiB 0.01% 7.67 KiB p

aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ minikube start
minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
E0502 03:07:40.095791 1759 start.go:846] api.Load failed for minikube: filestore "minikube": Docker machine "minikube" does not exist. Use "docker-machine ls" to list machines. Use "docker-machine create" to add a new one.
🌟 Using the docker driver based on existing profile
🔥 Starting "minikube" primary control-plane node in "minikube" cluster
📡 Pulling base image v0.0.50 ...
🔥 Creating docker container (CPUs=2, Memory=3072MB) ...
📡 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ minikube ip
192.168.49.2
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$
```

## Task 4: Scale the Deployment

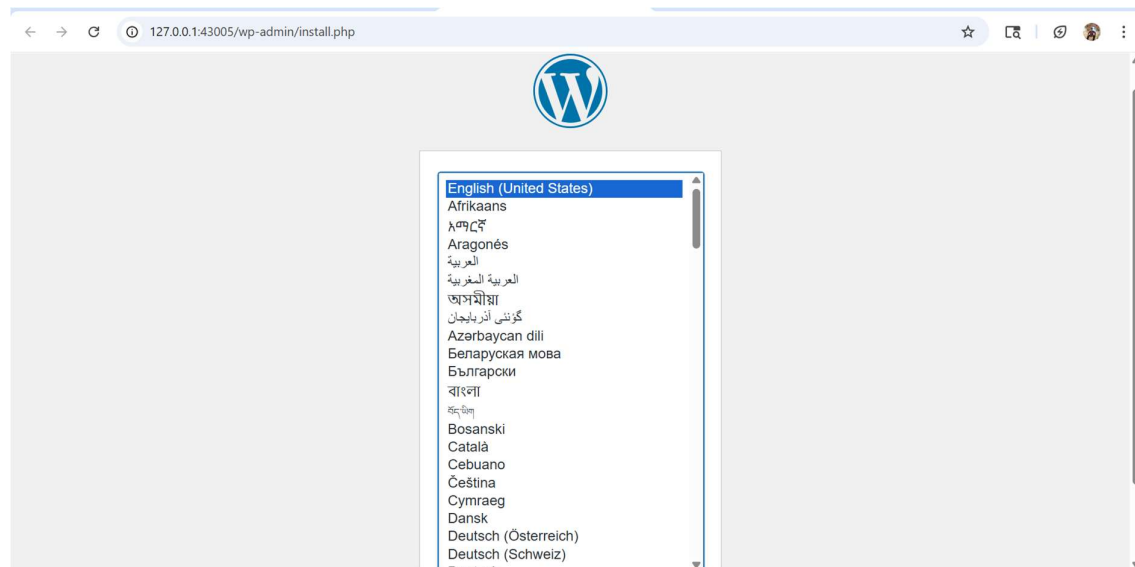
Increase the number of pods from 2 to 4:

```
kubectl scale deployment wordpress --replicas=4
```

Verify:

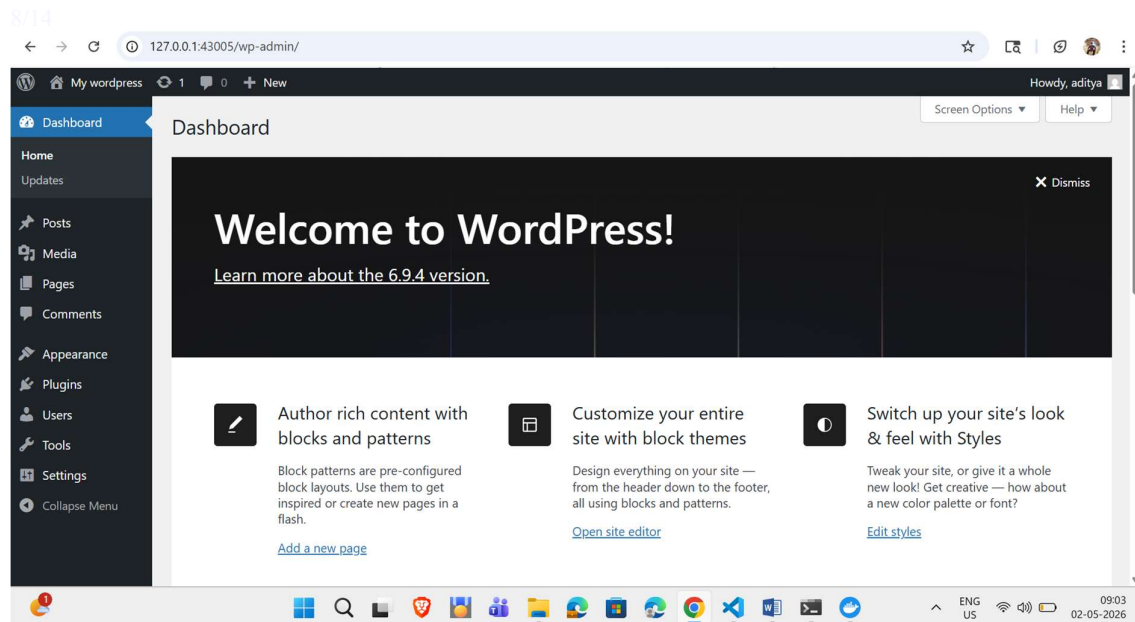
```
kubectl get pods
```

You should now see 4 running pods.



```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-59b756d576-pbbxh             1/1     Running   0           3m59s
wordpress-7bcdddf4d-gk65z          1/1     Running   0           2m43s
wordpress-7bcdddf4d-vdfh5          1/1     Running   0           11s
wordpress-7bcdddf4d-vl5xk          1/1     Running   0           2m43s
wordpress-7bcdddf4d-w9zb7          1/1     Running   0           11s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$
```

**Why scale?** More traffic → more copies → better performance.



## Task 5: Self-Healing Demonstration

Kubernetes automatically replaces failed pods.

Delete one pod manually:

```
# First, get pod names
kubectl get pods

# Delete one (replace <pod-name>)
kubectl delete pod <pod-name>
```

Now check pods again:

```
kubectl get pods
```

You will still see 4 pods — the deleted one was **automatically recreated**.

```
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl delete pod wordpress-7bcdddf4d-gk65z
pod "wordpress-7bcdddf4d-gk65z" deleted from default namespace
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
mysql-59b756d576-pbbxh             1/1     Running             0           5m52s
wordpress-7bcdddf4d-8bftl          0/1     ContainerCreating   0           6s
wordpress-7bcdddf4d-vdfh5          1/1     Running             0           2m4s
wordpress-7bcdddf4d-vl5xk          1/1     Running             0           4m36s
wordpress-7bcdddf4d-w9zb7          1/1     Running             0           2m4s
aditya@Aditya:/mnt/c/Users/Arundun/Desktop/Containers/Container/LAB/lab12$
```

**Why?** The deployment ensures the desired number (4) is always running.

9/14

## PART C – Swarm vs Kubernetes (Simple Comparison)

|              |              |                            |
|--------------|--------------|----------------------------|
| Feature      | Docker Swarm | Kubernetes                 |
| Setup        | Very easy    | More complex               |
| Scaling      | Basic        | Advanced (auto-scaling)    |
| Ecosystem    | Small        | Huge (monitoring, logging) |
| Industry use | Rare         | Standard                   |

**Verdict:** *Learn Kubernetes — it's what companies use.*

10/14

## PART D – Advanced Lab: Real Cluster with kubeadm

**Question:** Can we set up a real Kubernetes cluster?

**Yes — using kubeadm.** This is closer to a production setup.

### Lab Requirements

- 2 or 3 virtual machines (e.g., VirtualBox, VMware)
- Ubuntu 22.04 or 24.04
- Each VM: 2+ CPU, 2+ GB RAM

### High-Level Steps

#### Step 1: Install kubeadm, kubelet, kubectl on all nodes

```
# Update system
sudo apt update

# Install required packages
sudo apt install -y apt-transport-https ca-certificates curl

# Add Kubernetes signing key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg

# Add Kubernetes repository
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ .' | sudo tee /etc/apt/sources.list.d/kubernetes.list

# Install kubeadm, kubelet, kubectl
sudo apt update
sudo apt install -y kubeadm kubelet kubectl

# Hold versions to prevent auto-update
sudo apt-mark hold kubeadm kubelet kubectl
```

#### Step 2: Initialize the control plane (master node)

Run only on the **master node**:

```
sudo kubeadm init
```

This command:

- Sets up the control plane
- Generates a token for worker nodes to join
- Takes 2–3 minutes

#### Step 3: Set up kubeconfig (to use kubectl)

```
# Create .kube directory for your user
mkdir -p $HOME/.kube

# Copy admin config
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config

# Fix permissions
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Now you can run `kubectl get nodes` to see the master.

#### Step 4: Install a network plugin (required for pods to communicate)

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

Wait 1–2 minutes for Calico to start.

#### Step 5: Join worker nodes

On the **master**, after `kubeadm init`, you'll see a join command like:

```
kubeadm join 192.168.1.100:6443 --token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:...
```

Run **that exact command** on each worker node.

#### Step 6: Verify the cluster

On the master node:

```
kubectl get nodes
```

##### Expected output:

| NAME    | STATUS | ROLES         | AGE | VERSION |
|---------|--------|---------------|-----|---------|
| master  | Ready  | control-plane | 5m  | v1.29.0 |
| worker1 | Ready  | <none>        | 2m  | v1.29.0 |
| worker2 | Ready  | <none>        | 2m  | v1.29.0 |

11/14

### Teaching Insight: When to Use Which Tool

| Tool            | Best for                       |
|-----------------|--------------------------------|
| <b>k3d</b>      | Quick learning on your laptop  |
| <b>Minikube</b> | Single-node cluster testing    |
| <b>kubeadm</b>  | Real, production-style cluster |

12/14

### Summary of Commands (Cheat Sheet)

| Goal               | Command                                                         |
|--------------------|-----------------------------------------------------------------|
| Apply a YAML file  | <code>kubectl apply -f file.yaml</code>                         |
| See all pods       | <code>kubectl get pods</code>                                   |
| See all services   | <code>kubectl get svc</code>                                    |
| Scale a deployment | <code>kubectl scale deployment &lt;name&gt; --replicas=N</code> |
| Delete a pod       | <code>kubectl delete pod &lt;pod-name&gt;</code>                |
| See all nodes      | <code>kubectl get nodes</code>                                  |

13/14

### End of Experiment

You have now:

- Understood basic Kubernetes concepts
- Deployed WordPress using a Deployment and Service
- Scaled and tested self-healing
- Learned how to build a real cluster with kubeadm

**Next step:** Try deploying your own custom app (e.g., a simple Node.js or Python app).

14/14